

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

(повне найменування назва факультету)

Кафедра дискретного аналізу та інтелектуальних систем

(повна назва кафедри)

Курсова робота

РОЗРОБКА СИСТЕМИ ДЛЯ АВТОМАТИЧНОГО ВИЯВЛЕННЯ ФЕЙКОВИХ НОВИН

Виконав: студент групи ПМі-33с
спеціальності

122 «Комп'ютерні науки»

(шифр і назва спеціальності)

	<u>Яценко В.Т.</u>
	(підпис) (прізвище та ініціали)
Керівник	<u>Пелюшкевич О. В.</u>
	(підпис) (прізвище та ініціали)
Рецензент	<u></u>
	(підпис) (прізвище та ініціали)

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ І ТЕРМІНІВ	3
ВСТУП	5
1 ПОСТАНОВКА ЗАДАЧІ ТА ОГЛЯД НАЯВНИХ РІШЕНЬ.....	8
1.1 Огляд наявних рішень.....	8
1.2 Постановка задачі.....	9
1.3 Проєктування вимог.....	10
2 ВИКОРИСТАНІ ТЕХНОЛОГІЇ ТА АЛГОРИТМИ.....	15
2.1 Математичні основи попередньої обробки та векторизації тексту..	15
2.2 Базові моделі машинного навчання.....	16
2.3 Архітектури глибокого навчання на основі трансформерів	17
2.4 Програмний інструментарій та засоби розробки.....	21
3 РЕАЛІЗАЦІЯ СИСТЕМИ ТА РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ	23
3.1 Загальна архітектура системи	23
3.2 Формування та аналіз наборів даних	25
3.3 Реалізація базових моделей та результати	27
3.4 Реалізація моделей на основі mBERT та результати	31
3.5 Розгортання вебдодатка	37
3.6 Огляд роботи системи.....	37
3.7 Тестування системи та аналіз помилок.....	41
ВИСНОВКИ.....	45
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	47

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ І ТЕРМІНІВ

Accuracy – точність класифікації, частка правильно класифікованих зразків

Baseline – початкова модель з простою архітектурою, що використовується як точка відліку для порівняння з більш складними рішеннями

BERT – Bidirectional Encoder Representations from Transformers — двонаправлена трансформерна модель для отримання контекстуальних векторних представлень тексту

Data-Centric AI – підхід до розробки штучного інтелекту, де головна увага приділяється покращенню якості та структури даних, а не ускладненню архітектури моделі

Embedding – векторне представлення; відображення слів або токенів у щільний числовий вектор у просторі семантичних ознак

F1-score – гармонійне середнє між Precision та Recall

GELU – Gaussian Error Linear Unit — більш плавна версія ReLU, базовий стандарт для трансформерів

mBERT – Multilingual BERT — багатомовна версія моделі BERT

MLM – Masked Language Modeling — задача передтренування BERT: модель передбачає випадково замасковані токени на основі двобічного контексту

NLP – Natural Language Processing — обробка природної мови; галузь штучного інтелекту, що займається автоматичним аналізом та генерацією текстів

NSP – Next Sentence Prediction — друга задача передтренування BERT

Precision – точність, частка правильно визначених позитивних прикладів серед усіх визначених як позитивні

Recall – повнота, частка правильно визначених позитивних прикладів серед усіх фактично позитивних

ReLU – Rectified Linear Unit — функція, яка пропускає додатні числа без змін, а всі від'ємні перетворює на нуль

Self-Attention – механізм самоуваги; механізм, за якого кожен токен послідовності зважено враховує всі інші токени при формуванні свого векторного представлення

Softmax – функція, яка перетворює набір чисел на ймовірності (від 0 до 1), сума яких завжди дорівнює одиниці

SVM – Support Vector Machine — метод опорних векторів; алгоритм машинного навчання для класифікації, що знаходить оптимальну гіперплощину розділення класів

TF-IDF – Term Frequency — Inverse Document Frequency — статистична міра для оцінки важливості слова в документі, що є частиною колекції або корпусу

Векторизація – процес перетворення текстових даних у числовий формат

Лематизація – процес приведення слів до їх базової форми (леми)

Токенізація – процес розбиття тексту на окремі одиниці (токени), зазвичай слова або символи

Трансформер – архітектура нейронної мережі на основі механізму самоуваги, що обробляє всі токени послідовності паралельно без використання рекурентності

ВСТУП

Сучасний інформаційний простір характеризується безпрецедентними обсягами та швидкістю поширення даних. Соціальні мережі, онлайн–медіа та месенджери стали основними джерелами новин для значної частини населення, що створює як нові можливості для доступу до знань, так і суттєві виклики у контексті оцінки якості та достовірності контенту. По–перше, традиційні механізми перевірки інформації не встигають за темпами її поширення в цифровому середовищі. По–друге, поряд із достовірною інформацією цими ж каналами активно поширюється дезінформація, маніпулятивний контент та фейкові новини. Це явище становить серйозну загрозу для суспільної стабільності, демократичних процесів та національної безпеки, особливо в умовах гібридних війн та цілеспрямованих інформаційних атак. За даними дослідження Reuters Institute Digital News Report 2023[1], яке охоплює 46 країн світу, 56% користувачів стурбовані тим, що не можуть відрізнити правдиву інформацію від фейкової в інтернеті. В Україні ситуація є особливо критичною через триваючу інформаційну війну, що супроводжує військову агресію, де дезінформація використовується як інструмент гібридної війни для підризу довіри до державних інституцій та міжнародної підтримки. Водночас, зростаюча роль генеративних моделей штучного інтелекту у створенні правдоподібних, але фактично хибних повідомлень ускладнює ситуацію, роблячи фейкові новини все складнішими для розпізнавання людиною.

Сучасний стан проблеми розкривається в низці викликів, таких як обмежена масштабованість традиційного фактчекінгу, який виконується експертами вручну, відсутність єдиних стандартів перевірки достовірності інформації в україномовному сегменті, та необхідність швидкої реакції на інформаційні атаки. Традиційні методи верифікації новин часто не встигають за темпами генерації та поширення неправдивого контенту. Ці проблеми підкреслюють необхідність розвитку нових, автоматизованих підходів до виявлення фейкових новин, які б забезпечували швидкість, точність та масштабованість аналізу.

Актуальність даної роботи полягає у необхідності розробки та впровадження нових технологічних рішень, спрямованих на автоматизацію процесів виявлення недостовірної інформації. Технології машинного навчання та обробки природної мови можуть стати відповіддю на вищезгадані проблеми, гарантуючи швидку та ефективну фільтрацію інформаційних потоків. Такі системи дозволяють створювати моделі, що розрізняють характеристики правдивих та фейкових повідомлень на основі великих обсягів даних, сприяючи підвищенню стійкості суспільства до дезінформації.

Мета даної роботи полягає у розробці та реалізації моделей машинного навчання для автоматичного виявлення фейкових новин у текстових даних. Ця система має на меті забезпечити точну класифікацію новин за достовірністю та тематикою, одночасно акцентуючи увагу на доступності та зручності у використанні для кінцевих користувачів через вебінтерфейс.

Розробка системи виявлення фейкових новин передбачає комплексний аналіз існуючих підходів та методів, виявлення прогалин та можливостей для вдосконалення. Також вона включає оцінку різних технологій машинного навчання, зокрема методів векторизації тексту, алгоритмів класифікації та архітектур нейронних мереж, на основі точності, швидкості роботи та придатності для україномовних даних. Вибір цих технологій базується на вимогах і цілях проєкту. Створення системи включає етапи збору та попередньої обробки даних, побудови та тренування моделей, тестування та розгортання. Користувацький інтерфейс також детально описується, включаючи функціональні можливості та принципи взаємодії з користувачем, щоб забезпечити інтуїтивно зрозумілу та доступну систему для всіх.

Основними завданнями даної курсової роботи є дослідження існуючих підходів до автоматичного виявлення фейкових новин та тематичної класифікації тексту, збір та підготовка корпусу україномовних новин, статистичний аналіз текстових даних, розробка базової моделі на основі логістичної регресії з використанням TF-IDF векторизації, дослідження більш складних архітектур, зокрема нейронних мереж, оцінка ефективності розроблених моделей за

допомогою метрик якості (accuracy, precision, recall, F1-score), та створення вебдодатку для демонстрації роботи моделей.

Результатом роботи є вебдодаток для виявлення фейкових новин, який є зручною платформою, здатною аналізувати текстовий контент та надавати прогноз щодо його достовірності. Систему можна застосовувати в медіа-моніторингу, на соціальних платформах, в освітніх закладах та для особистого використання. Медіа-компанії можуть використовувати її для попередньої фільтрації контенту та маркування потенційно недостовірної інформації, а освітні заклади – для підвищення медіаграмотності студентів. Дослідники медіа можуть застосовувати систему для аналізу інформаційних потоків та вивчення механізмів поширення дезінформації. Це ключовий момент, який допомагає суспільству протистояти інформаційним маніпуляціям та розвивати критичне мислення при споживанні новин.

1 ПОСТАНОВКА ЗАДАЧІ ТА ОГЛЯД НАЯВНИХ РІШЕНЬ

1.1 Огляд наявних рішень

Сучасні підходи до верифікації інформації та виявлення дезінформації можна класифікувати за рівнем автоматизації, цільовою аудиторією та використаними технологіями. Глобально їх поділяють на традиційний експертний фактчекінг, інтерактивні інструменти та системи на базі машинного навчання.

Традиційний фактчекінг базується на методології відкритої розвідки та ручному аналізі джерел журналістами-дослідниками. На глобальному рівні стандартом агрегації таких перевірок є Fact Check Explorer [2] від компанії Google – пошукова система, що збирає результати від понад 100 міжнародних організацій за допомогою мікророзмітки ClaimReview та надає API для інтеграції у сторонні сервіси [3]. Серед міжнародних платформ виділяється американський PolitiFact [8], який використовує систему наочних рейтингів для оцінки достовірності політичних заяв. Головним недоліком цього підходу є часовий лаг: ручна перевірка потребує годин або навіть днів, тоді як фейкова новина поширюється мережею за лічені хвилини.

В умовах інформаційної війни в Україні провідну роль відіграють спеціалізовані організації. Проєкт StopFake [4], заснований у 2014 році, фокусується на спростуванні російської пропаганди у ЗМІ. VoxCheck [5] спеціалізується на фактчекінгу заяв українських політиків та економічних маніпуляцій. Ініціатива «Перевірка» від Gwara Media [7] функціонує як Telegram-бот, де команда аналітиків обробляє підозрілі посилання у напівавтоматичному режимі. Попри високу точність та експертність, усі ці проєкти стикаються з проблемою масштабованості через обмеженість людського ресурсу.

Паралельно розвиваються інтерактивні рішення для підвищення медіаграмотності. Зокрема, FakeCheck.in.ua [6] пропонує користувачам пройти чек-лист оцінки матеріалу, після чого алгоритм видає ймовірнісну оцінку достовірності. Однак такий підхід є евристичним і покладається на суб'єктивне сприйняття читача, а не на аналіз даних.

Спроби повної автоматизації верифікації за допомогою методів обробки природної мови (NLP) представлені переважно академічними ініціативами. Дослідницький проєкт Університету Саймона Фрейзера [9] та подібні роботи демонструють ефективність класичного машинного навчання – зокрема, алгоритмів SVM з TF-IDF векторизацією – для аналізу англomовних новин. Проте такі рішення різко втрачають ефективність при роботі з україномовним контентом через складну морфологію мови та специфіку локальних маніпулятивних патернів, які неможливо виявити за допомогою базового словникового аналізу. Вирішення цієї проблеми вимагає переходу до архітектур на базі трансформерів, зокрема багатомовних моделей класу BERT, здатних розуміти глибинний контекст тексту.

1.2 Постановка задачі

У курсовій роботі було поставлено за мету розробити систему автоматичного виявлення фейкових новин для україномовного контенту з використанням методів машинного навчання та обробки природної мови, з огляду на складну морфологію та лексико–семантичні особливості української мови.

Система має забезпечити бінарну класифікацію новин (правдива/фейкова) та додаткову тематичну категоризацію контенту, надаючи користувачам швидкий та об'єктивний інструмент для оцінки достовірності інформації через зручний вебінтерфейс. Для досягнення поставленої мети необхідно вирішити наступні задачі:

- 1) Збір та формування якісного корпусу україномовних новин з верифікованими мітками достовірності (правдива/фейкова) та тематичними категоріями, що є критично важливим етапом через загальний дефіцит відкритих та структурованих україномовних датасетів.
- 2) Дослідження та імплементація методів попередньої обробки текстових даних (токенізація, видалення стоп–слів, лематизація та векторизація) з обов'язковою адаптацією алгоритмів до багатой словозміни, складного словотворення та вільного синтаксису, притаманних українській мові.

- 3) Розробка базової моделі класифікації на основі логістичної регресії з використанням TF–IDF векторизації для встановлення baseline метрик якості та виявлення маркерної лексики.
- 4) Дослідження та імплементація сучасних архітектур машинного навчання на базі трансформерів (зокрема, багатомовних моделей класу mBERT), які здатні розуміти глибинний контекст тексту та виявляти специфічні маніпулятивні патерни україномовного інфопростору, нівелюючи проблему багатозначності слів.
- 5) Проведення порівняльного аналізу розроблених моделей за метриками accuracy, precision, recall та F1–score на тестовій вибірці.
- 6) Розробка вебдодатка з інтуїтивним користувацьким інтерфейсом для демонстрації роботи моделей, що дозволяє користувачам вводити текст новини та отримувати прогноз щодо її достовірності та тематики.

1.3 Проєктування вимог

Вимоги до програмного забезпечення визначають характеристики та обмеження системи, які необхідно реалізувати для досягнення поставленої мети. Вони поділяються на функціональні, що описують конкретні можливості та поведінку системи, та нефункціональні, що визначають якісні характеристики та обмеження, за яких система має функціонувати. Чітко сформульовані вимоги є основою для проєктування архітектури, розробки, тестування та оцінки успішності проєкту.

Для візуалізації функціональних вимог системи розроблено UML діаграму прецедентів, яка представлена на рисунку 1.1.

Діаграма прецедентів відображає основні сценарії взаємодії користувача з системою виявлення фейкових новин. Архітектура системи логічно розділена на дві взаємопов'язані підсистеми: введення та обробка (ліва частина діаграми) та аналіз результатів (права частина діаграми).

Користувач виступає єдиним актором системи та має доступ до всього функціоналу через вебінтерфейс. Взаємодія користувача з системою відбувається

через десять основних прецедентів, які охоплюють повний цикл роботи з новинним контентом – від введення даних до отримання результатів аналізу.

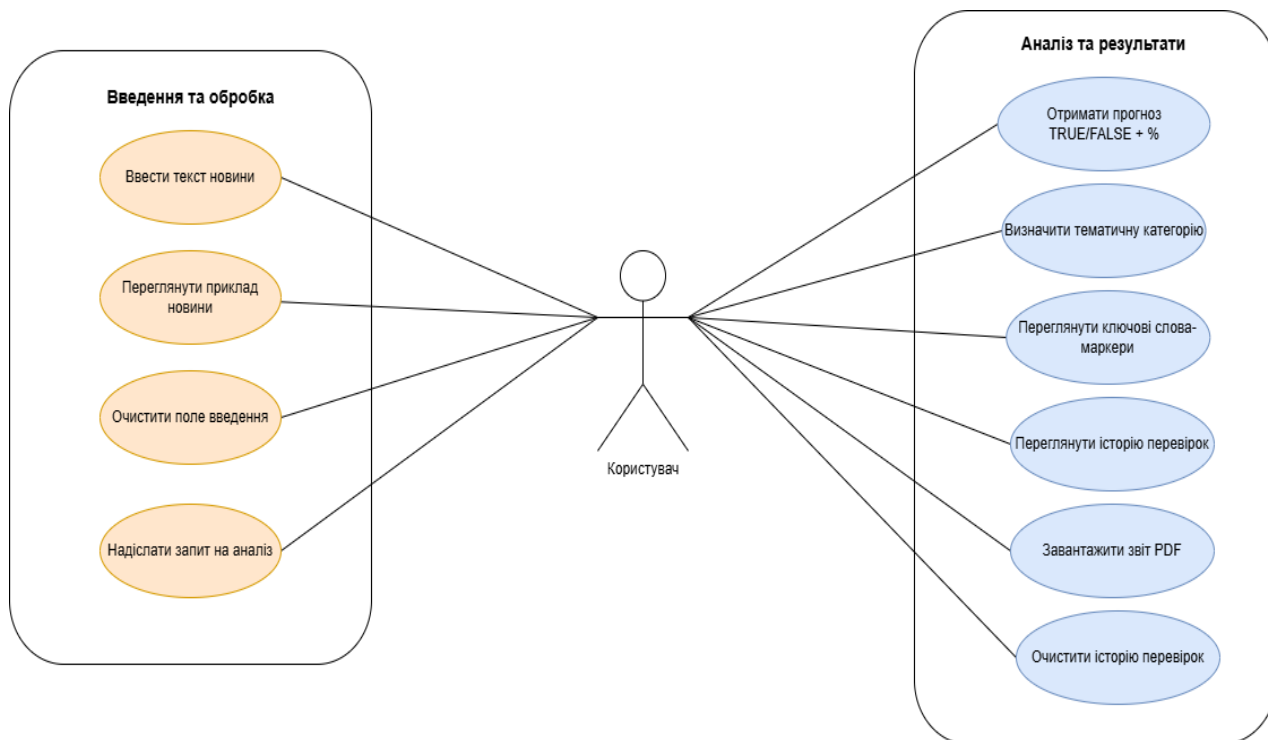


Рисунок 1.1 – Діаграма прецедентів

Функціональні вимоги визначають конкретні функції, операції та можливості, які система має надавати користувачам. Вони описують, що саме система повинна робити у відповідь на певні входні дані або в певних ситуаціях.

Система має забезпечувати наступні функціональні можливості:

- 1) Бінарна класифікація новин за критерієм достовірності (правдива/фейкова) з точністю не менше 80% на тестовій вибірці.
- 2) Тематична категоризація контенту для забезпечення контекстуального аналізу та можливості фільтрації результатів за предметними областями.
- 3) Обробка україномовного тексту з підтримкою різних стилів написання (формальний/неформальний, новинний/блоговий формат).
- 4) Попередня обробка вхідного тексту, що включає: токенизацію українського тексту, нормалізацію (приведення до нижнього

регістру), видалення стоп–слів на основі українського словника, лематизацію, та векторизацію.

- 5) Вебінтерфейс має надавати можливість введення тексту новини через текстове поле з підтримкою копіювання/вставки контенту.
- 6) Відображення результатів аналізу для кожної новини у форматі карточки або таблиці з чіткою індикацією: відсоток ймовірності фейковості (0–100%), визначена тематична категорія.
- 7) Збереження історії перевірених новин у сесії користувача для можливості порівняння результатів.
- 8) Експорт результатів аналізу у форматі CSV або Excel з включенням наступних полів: оригінальний текст новини, прогноз достовірності (FAKE/TRUE), ймовірність фейковості (%), тематична категорія, дата та час перевірки.
- 9) Адаптивний інтерфейс, що коректно відображається на різних пристроях (десктоп, планшет, смартфон).

Нефункціональні вимоги визначають якісні характеристики системи, такі як продуктивність, надійність, безпека, масштабованість та зручність використання. Вони описують не що система робить, а як вона це робить, встановлюючи стандарти якості та обмеження, за яких система має функціонувати.

Система має відповідати наступним нефункціональним вимогам:

1) Продуктивність та масштабованість:

- Час відгуку вебінтерфейсу – не більше 1 секунди при стандартному інтернет–з'єднанні.
- Архітектура має забезпечувати можливість горизонтального масштабування для обробки зростаючих обсягів даних.

2) Надійність та доступність:

- У разі виникнення помилки обробки (некоректний формат даних, занадто короткий текст, технічний збій) система має відображати зрозуміле повідомлення для користувача без розкриття внутрішніх технічних деталей або трасування стеку помилок.

- Передбачити обробку виключних ситуацій (некоректний формат вхідних даних, обрив з'єднання) з коректним інформуванням користувача.
- Система має зберігати працездатність при роботі з текстами, що містять помилки правопису, опечатки, повтори символів або нестандартні розділові знаки без критичних збоїв.

3) Зручність використання:

- Інтерфейс має бути інтуїтивно зрозумілим для користувачів без технічної підготовки або попереднього досвіду роботи з системами машинного навчання.
- Інтерактивні елементи мають мати зрозумілі підказки та описи.
- Швидкий час навчання для роботи з системою новому користувачу.
- Інтерфейс має бути українською мовою з чіткими формулюваннями всіх елементів без технічного жаргону.

4) Підтримка та розширюваність:

- Вихідний код має бути опублікований як open-source на платформі GitHub для можливості подальшого розвитку дослідниками та розробниками.
- Код має бути задокументований з коментарями для ключових функцій та класів згідно зі стандартом PEP 257 для Python docstrings.
- README файл у репозиторії має містити: детальні інструкції з встановлення залежностей, крок-за-кроком гайд з розгортання системи локально та на сервері, опис структури проекту та призначення кожного модуля.

5) Сумісність:

- Вебдодаток має коректно працювати у сучасних браузерях останніх двох версій.

- Підтримка роботи на операційних системах Windows, macOS, Linux.

6) Етичні вимоги та прозорість:

- Рішення моделей машинного навчання мають супроводжуватися поясненнями через відображення найбільш значущих слів та фраз, що вплинули на класифікацію.
- Система має включати чітко видиме застереження про те, що результати є автоматичними прогнозами моделей машинного навчання і можуть містити помилки, рекомендуючи додаткову перевірку інформації через офіційні джерела для критично важливих ситуацій.
- Результати класифікації не мають представлятися як абсолютна істина, а лише як допоміжний інструмент для користувача у процесі критичного осмислення інформації.

Сформульовані вимоги слугують основою для проєктування архітектури системи, вибору технологій та методів машинного навчання, а також для подальшого тестування та валідації розробленого рішення. Дотримання цих вимог забезпечить створення функціональної, надійної та зручної системи для автоматичного виявлення фейкових новин, яка відповідатиме потребам українських користувачів у боротьбі з дезінформацією в сучасному цифровому інформаційному просторі.

2 ВИКОРИСТАНІ ТЕХНОЛОГІЇ ТА АЛГОРИТМИ

2.1 Математичні основи попередньої обробки та векторизації тексту

Машинні алгоритми не здатні безпосередньо аналізувати символічні рядки, тому процес виявлення маніпуляцій у новинах починається з трансформації тексту у математичні структури. Цей етап включає кілька послідовних кроків: токенизацію, очищення від стоп-слів та лематизацію.

Токенизація відповідає за розбиття суцільного тексту на окремі слова. Після цього з тексту вилючаються стоп-слова – елементи мови, що не несуть важливого семантичного навантаження. Для зменшення розмірності простору ознак застосовується лематизація – зведення всіх слів до їхньої словникової форми.

Після морфологічної обробки застосовується векторизація тексту за мірою TF-IDF, що складається з двох компонентів. Термінна частота (TF) визначає відносну частоту слова t у документі d

$$TF(t, d) = \frac{n_t}{\sum_k n_k} \quad (2.1)$$

де n_t – кількість входжень слова t , а знаменник – загальна кількість слів у документі. Обернена частота документа (IDF) знижує вагу слів, що зустрічаються майже в усіх текстах, і підвищує вагу специфічних термінів:

$$IDF(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right) \quad (2.2)$$

де $|D|$ – розмір корпусу, а знаменник – кількість документів, що містять слово t . Результируюча вага слова у документі обчислюється як:

$$W(t, d) = TF \times IDF \quad (2.3)$$

Це дозволяє виділити слова-маркери, характерні саме для фейкових або правдивих новин.

На відміну від TF-IDF, архітектури на базі трансформерів (зокрема mBERT) використовують власний механізм токенизації – алгоритм WordPiece, який розбиває складні слова на підслова. Це дозволяє моделі ефективно працювати з багатою морфологією української мови без створення словників надмірного розміру. На етапі токенизації кожна послідовність доповнюється спеціальними маркерами:

[CLS] – акумулює загальний контекст для класифікації, [SEP] – позначає межі речень, а маски уваги вказують моделі на ігнорування токенів-заповнювачів.

2.2 Базові моделі машинного навчання

Для вирішення поставлених завдань архітектура розроблюваної системи була розділена на два незалежні класифікаційні модулі. Як базова лінія для порівняння з нейромережевими підходами були застосовані класичні лінійні алгоритми.

Першим алгоритмом, який відповідає за модуль детектування достовірності (визначення ймовірності фейковості новини), обрано логістичну регресію [10]. Цей алгоритм обчислює зважену суму входних ознак (значень TF-IDF) та пропускає результат через логістичну функцію (сигмоїду):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.4)$$

де z – лінійна комбінація ознак тексту та їхніх ваг. Згідно з формулою (2.4), функція нелінійно стискає будь-яке дійсне значення у діапазон від 0 до 1, що дозволяє інтерпретувати результат як статистичну ймовірність того, що новина є маніпулятивною.

Другим алгоритмом для модуля тематичної категоризації обрано метод опорних векторів [11] з лінійним ядром. Його завдання – знайти оптимальну гіперплощину у багатовимірному просторі ознак, яка найкраще розділяє тематичні класи з максимальним зазором (маржею). Для розв'язання проблеми багатокласової класифікації алгоритм застосовує стратегію «один проти всіх».

Оскільки реальні текстові дані рідко є ідеально лінійно роздільними, на практиці використовується алгоритм з м'яким зазором. Процес навчання моделі зводиться до розв'язання такої оптимізаційної задачі:

$$\min_{\omega, b, \xi} \left(\frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^n \xi_i \right) \quad (2.5)$$

де ω – вектор ваг моделі;

b – скалярне зміщення;

ξ_i – змінні послаблення, що дозволяють алгоритму ігнорувати незначний шум у даних;

C – параметр регуляризації (штраф за помилку), який контролює баланс між шириною зазору та кількістю допущених помилок на навчальній вибірці.

Після того, як алгоритм знаходить оптимальні параметри, класифікація нового невідомого тексту здійснюється шляхом обчислення знака вирішальної функції: якщо результат позитивний, новина відноситься до цільової категорії, якщо негативний – до інших.

2.3 Архітектури глибокого навчання на основі трансформерів

Класичні методи машинного навчання, зокрема логістична регресія та метод опорних векторів, працюють у рамках парадигми «мішок слів». У цьому підході текст розглядається як невпорядкований набір слів без урахування порядку, синтаксису та контексту. Наприклад, речення «Зеленський заявив, що Росія бреше» та «Росія заявила, що Зеленський бреше» матимуть однакове векторне представлення попри протилежний зміст. Це є критичним обмеженням для задачі виявлення фейкових новин, де ключову роль відіграє саме контекст.

Для подолання цього обмеження застосовуються моделі глибокого навчання на основі архітектури трансформера, які здатні враховувати взаємозв'язки між словами в межах усього речення. У даній роботі як основу системи детекції використано багатомовну модель mBERT.

Трансформер – це архітектура нейронної мережі, що використовує механізм самоуваги для обробки тексту. На відміну від рекурентних моделей, трансформер аналізує всі токени послідовності паралельно, що дозволяє ефективно враховувати залежності між словами незалежно від їхньої позиції в тексті. Механізм самоуваги передбачає обчислення трьох векторів для кожного токена: запиту (query), ключа (key) та значення (value). Вплив одного слова на інше визначається на основі взаємодії цих векторів із подальшою нормалізацією функцією softmax. Це дозволяє моделі формувати контекстуально залежні представлення тексту.

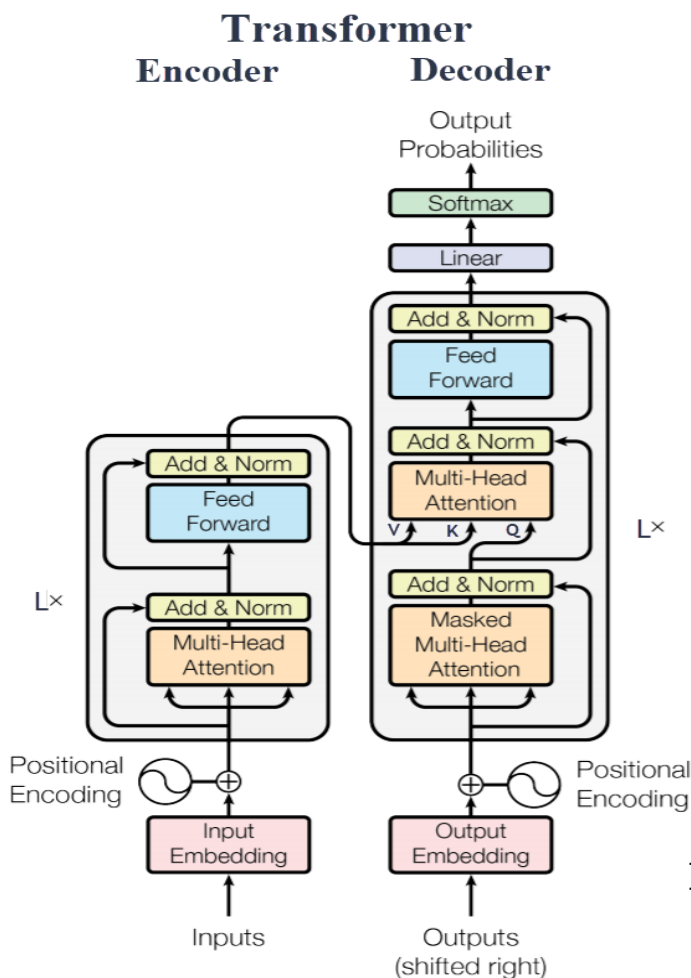


Рисунок 2.1 – Архітектура трансформерного енкодера (Vaswani et al., 2017)

[12]

На базі трансформерної архітектури побудовано модель BERT, яка використовує двонаправлене врахування контексту – тобто одночасно аналізує як ліву, так і праву частину речення. Це дозволяє отримувати більш точні та семантично насичені векторні представлення тексту порівняно з класичними підходами.

Для роботи з україномовними текстами у даній системі застосовано багатомовну версію моделі – mBERT. Вона попередньо навчена на корпусах 104 мов, що забезпечує ефективне перенесення знань між мовами та дозволяє обробляти українські тексти без необхідності створення окремого великого корпусу даних або словника.

BERT – модель попереднього навчання, запропонована Devlin et al. (2019) [13]. Її ключова відмінність від попередніх підходів полягає у двонаправленому

врахуванні контексту, що дозволяє одночасно обробляти лівий і правий контекст кожного токена. Навчання моделі базується на двох задачах: передбаченні випадково замаскованих 15% tokenів та аналізі послідовності речень.

Архітектура bert-base включає 12 блоків трансформерного енкодера з прихованим розміром та 12 головами уваги, що сумарно складає близько 110 мільйонів параметрів. Токенізація здійснюється за алгоритмом WordPiece, який розбиває рідкісні слова на частіші підслова, забезпечуючи гнучкість словника. Спеціальний токен [CLS] на початку послідовності акумулює контекстуальну інформацію всього тексту для подальшої класифікації.

mBERT є багатомовним розширенням, навченим на корпусах 104 мов, включаючи українську. Словник моделі розширено до 119 547 tokenів для охоплення різних алфавітів. Завдяки здатності до перенесення знань між мовами (cross-lingual transfer), верифікований дослідженнями Pires et al. (2019) [14], mBERT ефективно працює з україномовним контентом без необхідності у великих вузькоспеціалізованих корпусах.

Трансферне навчання – це підхід, за якого модель, попередньо навчена на великому загальному корпусі, адаптується до вузької цільової задачі. Для трансформерних архітектур виділяють дві основні стратегії: повне донавчання та екстракцію ознак. Перша передбачає оновлення всіх параметрів моделі, включаючи ваги базового енкодера. Це забезпечує найкращі результати при достатньому обсязі даних, проте вимагає значних ресурсів і несе ризик «катастрофічного забування» – деградації загальних мовних знань. Стратегія екстракції ознак полягає у заморожуванні ваг енкодера, де навчання обмежується лише класифікаційним шаром, що є обчислювально ефективним для малих датасетів.

У даній роботі застосована стратегія повного донавчання: всі 177 мільйонів параметрів моделі, включаючи ваги mBERT-енкодера та класифікаційного шару, оновлювалися під час навчання. Такий підхід є обґрунтованим з огляду на розмір використаного датасету та гарантує максимальну адаптацію мовних представлень до специфіки україномовних фейкових новин. Проведене порівняльне дослідження

трьох архітектур класифікаційних голів за ідентичних умов дозволяє ізолювати вплив топології верхніх шарів від впливу самого енкодера.

Класифікаційна голова – це нейромережевий модуль поверх базового енкодера, що перетворює векторне представлення тексту на передбачення класу. Обрана архітектура визначає здатність системи моделювати складні нелінійні залежності та метод агрегації ознак із векторного простору. У роботі досліджено три підходи:

Лінійна класифікація: найпростіша пряма проєкція вектора [CLS] через один повнозв'язний шар. Метод має мінімальну кількість параметрів та високу інтерпретованість, проте лінійна гіперплощина може бути недостатньою для виявлення складних маніпулятивних патернів. Для регуляризації застосовується Dropout [15].

Багатошаровий перцептрон (MLP) із пакетною нормалізацією: додавання проміжного прихованого шару з функцією активації ReLU для апроксимації нелінійних рішачих меж. Ключовим елементом є Batch Normalization, що стабілізує розподіл активацій та пришвидшує збіжність моделі.

Пулінг на основі механізму уваги (Attention Pooling): альтернативна агрегація через зважене усереднення прихованих станів усіх токенів замість використання лише [CLS]. Механізм визначає важливість кожного слова (наприклад, емоційно забарвлених маркерів), що робить модель виразнішою, але потребує більше даних для навчання. Для стабільності використовується Layer Normalization [17].

Порівняльний аналіз цих трьох архітектур при однакових умовах навчання дозволяє емпірично встановити, наскільки ускладнення топології класифікаційного шару впливає на здатність системи детектувати фейкові новини в україномовному датасеті. Конкретна реалізація, гіперпараметри та результати порівняння описані у розділі 3.

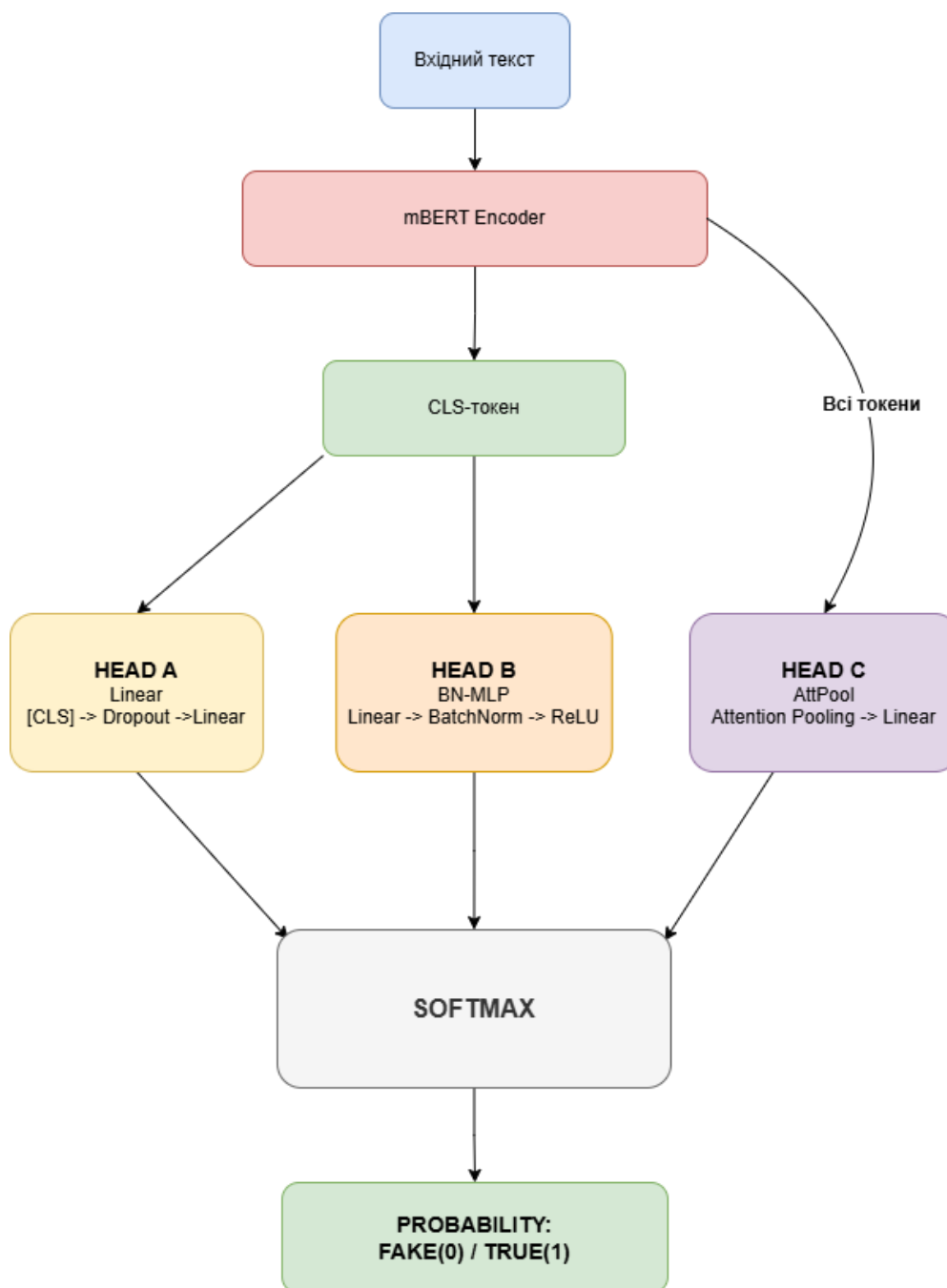


Рисунок 2.2 – Схема трьох досліджуваних архітектур класифікаційних голів поверх mBERT енкодера

2.4 Програмний інструментарій та засоби розробки

Програмна реалізація системи виконана мовою програмування Python, що є стандартом у галузі машинного навчання та обробки природної мови.

Для структурування та пакетної обробки наборів даних використано бібліотеку Pandas, а математичні операції реалізовано засобами NumPy [18].

Візуалізація результатів експериментів виконана за допомогою бібліотек Matplotlib та Seaborn [19], генерація хмар слів – за допомогою WordCloud.

Класичні алгоритми машинного навчання реалізовано засобами бібліотеки Scikit-learn [20]: векторизацію тексту виконує TfidfVectorizer, навчання базових моделей – LogisticRegression та LinearSVC, калібрування ймовірностей SVM – CalibratedClassifierCV. Сериалізацію класичних моделей забезпечують бібліотеки joblib та pickle.

Для морфологічного аналізу україномовних текстів та лематизації використано аналізатор PyMorphu3 [21], що дозволяє зводити слова до початкової форми з урахуванням багатой словозміни української мови.

Реалізацію нейромережових архітектур на основі трансформерів виконано засобами фреймворку PyTorch [22] та бібліотеки Hugging Face Transformers [13], яка надає готову реалізацію моделі bert-base-multilingual-cased з відповідним токенизатором. Прогрес навчання відстежувався за допомогою бібліотеки tqdm.

Серверну частину вебзастосунку реалізовано на мікрофреймворку Flask [23] з розширеннями Flask-Login для управління сесіями та Flask-SQLAlchemy для роботи з базою даних. Хешування паролів виконує бібліотека Werkzeug. Генерацію PDF-звітів забезпечує ReportLab [24], розгортання у продуктивному середовищі – сервер Gunicorn [25].

Усі експерименти з навчання моделей проводились у інтерактивному середовищі Jupyter Notebook та організовані у восьми послідовних ноутбуках відповідно до етапів дослідження.

3 РЕАЛІЗАЦІЯ СИСТЕМИ ТА РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ

3.1 Загальна архітектура системи

Розроблена система являє собою веб-застосунок, побудований на основі мікрофреймворку Flask, що реалізує багатомодельний підхід до аналізу текстів новин. Загальний процес обробки запиту – від введення тексту користувачем до відображення результатів – представлено на рисунку 3.1.

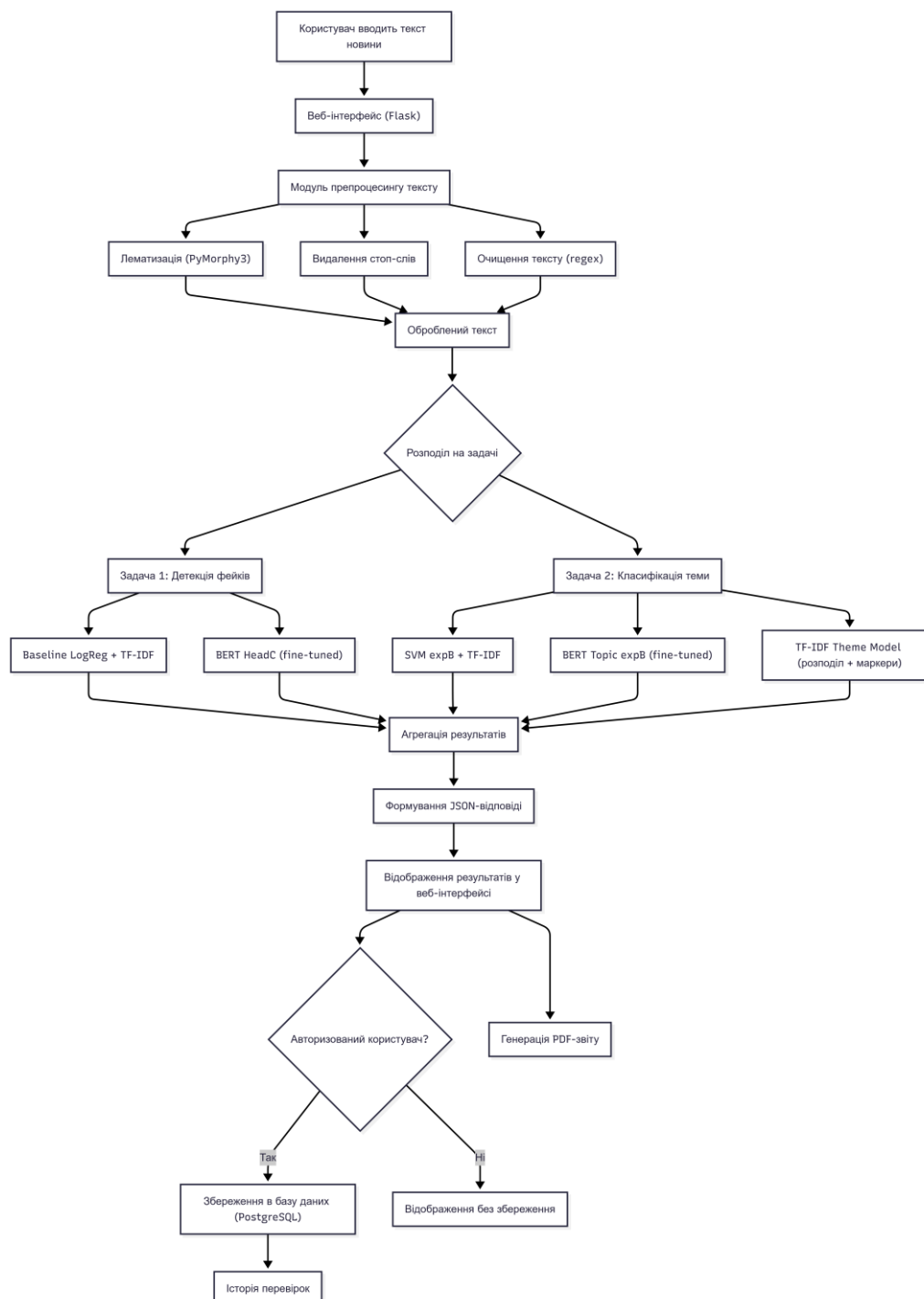


Рисунок 3.1 – Схема архітектури системи аналізу достовірності новин

Архітектура системи складається з шести функціональних блоків.

Блок введення даних. Користувач взаємодіє з системою через веб-інтерфейс, вводячи текст новини у текстове поле. Після подання запиту текст передається на серверну частину для подальшої обробки.

Блок препроцесингу тексту. Вхідний текст проходить послідовну обробку: приведення до нижнього регістру, очищення від спеціальних символів та HTML-тегів за допомогою регулярних виразів, видалення стоп-слів на основі українського словника та морфологічну лематизацію за допомогою бібліотеки PyMorphy3. Результатом є нормалізований текст, придатний для векторизації та класифікації.

Блок детекції фейкових новин. Цей блок відповідає за бінарну класифікацію тексту на категорії TRUE та FAKE. Паралельно працюють дві моделі: базова модель логістичної регресії на TF-IDF-векторах та основний класифікатор на базі BERT з класифікаційною головою Head C. Обґрунтування вибору архітектур та результати їхнього порівняння наведено у підрозділі 3.3.

Блок класифікації тематики. Цей блок визначає тематичну категорію новини з чотирьох класів: політика, спорт, бізнес та технології. Використовуються три моделі: SVM на TF-IDF-представленнях, BERT Topic expB та допоміжна TF-IDF Theme Model, що формує розподіл ймовірностей за темами та витягує ключові слова-маркери. Детальний опис та порівняння моделей наведено у підрозділі 3.4.

Блок агрегації результатів. Після обробки всіма моделями результати агрегуються у єдину JSON-структуру, що містить вердикт кожної моделі з відповідними ймовірностями, визначену тематичну категорію та ключові слова-маркери.

Блок збереження та візуалізації. Результати відображаються у веб-інтерфейсі у вигляді карток з кольоровою індикацією. Для авторизованих користувачів результати зберігаються у базу даних, а також доступна генерація PDF-звіту засобами бібліотеки ReportLab. Неавторизовані користувачі можуть використовувати систему у гостьовому режимі без збереження історії.

3.2 Формування та аналіз наборів даних

Для навчання та оцінки розроблених моделей було сформовано два незалежних датасети – окремо для задачі детекції фейкових новин та для задачі тематичної класифікації.

Перший датасет містить 21 738 зразків із збалансованим розподілом міток: 10 869 правдивих новин (TRUE) та 10 869 фейкових новин (FAKE). Правдиві новини зібрано з верифікованих українських медіа – Укрінформ, Українська правда, Суспільне та офіційні прес-служби державних органів. Фейкові новини отримано з фактчекінгових порталів StopFake.org, VoxCheck та розділу спростувань Укрінформу. Кожен зразок категорії FAKE є верифікованим – тобто інформацією, хибність якої підтверджена редакцією відповідного portalу. Збалансованість датасету унеможливорює зміщення моделі в бік одного з класів без додаткової компенсації.

Розбиття датасету здійснювалось по-різному залежно від типу моделі. Для класичних моделей застосовано співвідношення 80/20 зі стратифікацією: 17 382 зразки для навчання та 4 346 для тестування. Для моделей на основі BERT застосовано тричастинне розбиття: 70% – тренувальна вибірка (15 216 зразків), 15% – валідаційна (3 261 зразок) та 15% – тестова (3 261 зразок).

Аналіз датасету наведено на рисунку 3.2. Серед найбільш вживаних слів після лематизації переважають терміни, пов'язані з воєнною тематикою, політикою та державними інституціями, що відображає специфіку україномовного новинного контенту.

Другий датасет призначений для задачі тематичної класифікації та містить 50 000 зразків, рівномірно розподілених по п'яти категоріях: політика, спорт, новини, технології та бізнес – по 10 000 зразків на кожную. Новини зібрано з відкритих україномовних новинних агрегаторів та медіаархівів. Для кожного зразка використовувалась комбінація трьох полів – заголовка, основного тексту та тегів, що забезпечує повніше семантичне представлення при векторизації. Застосовано розбиття 80/20 зі стратифікацією для збереження рівномірного розподілу категорій у тренувальній та тестовій вибірках.

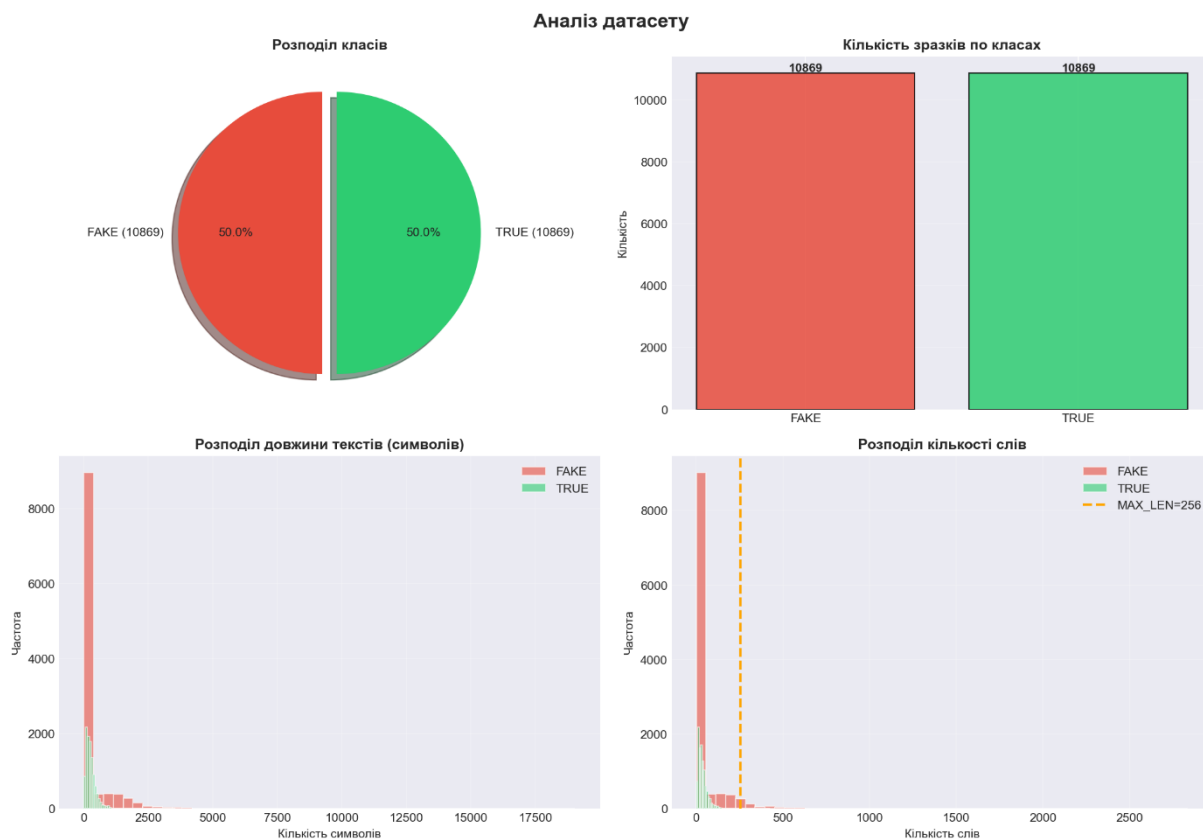


Рисунок 3.2 – Аналіз датасету фейкових та правдивих новин

Аналіз датасету тематичної класифікації наведено на рисунку 3.3. Як видно з хмари найчастіших слів (рисунку 3.4), серед найбільш вживаних лексем переважає загальна ділова та інформаційна лексика – «компанія», «людина», «команда», «зазначити», «повідомити». Висока частотність таких слів одночасно в різних категоріях свідчить про те, що статистичні методи типу TF-IDF є недостатньо ефективними для розмежування тем, оскільки не враховують глибинний контекст.

Аналіз датасету тематичної класифікації

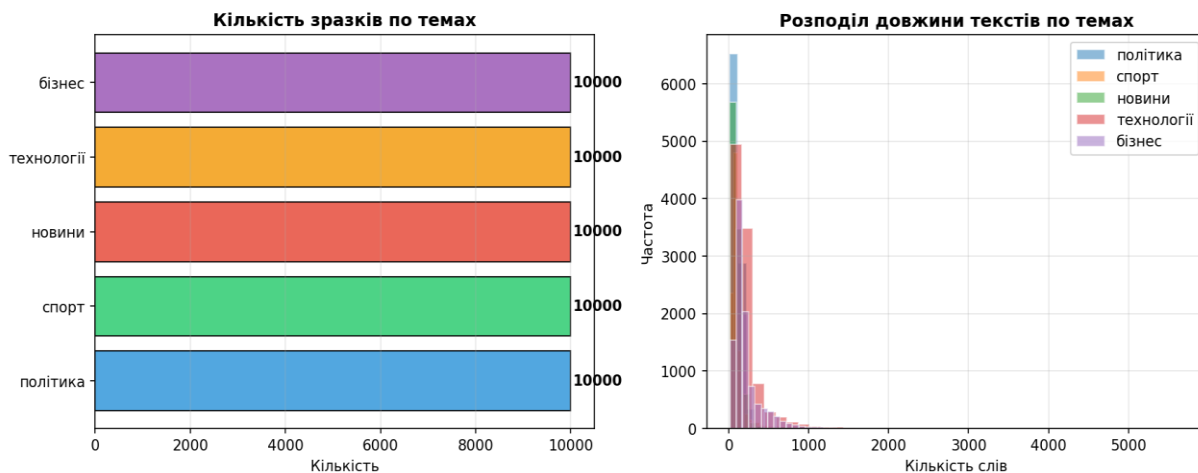


Рисунок 3.3 – Аналіз датасету тематичної класифікації

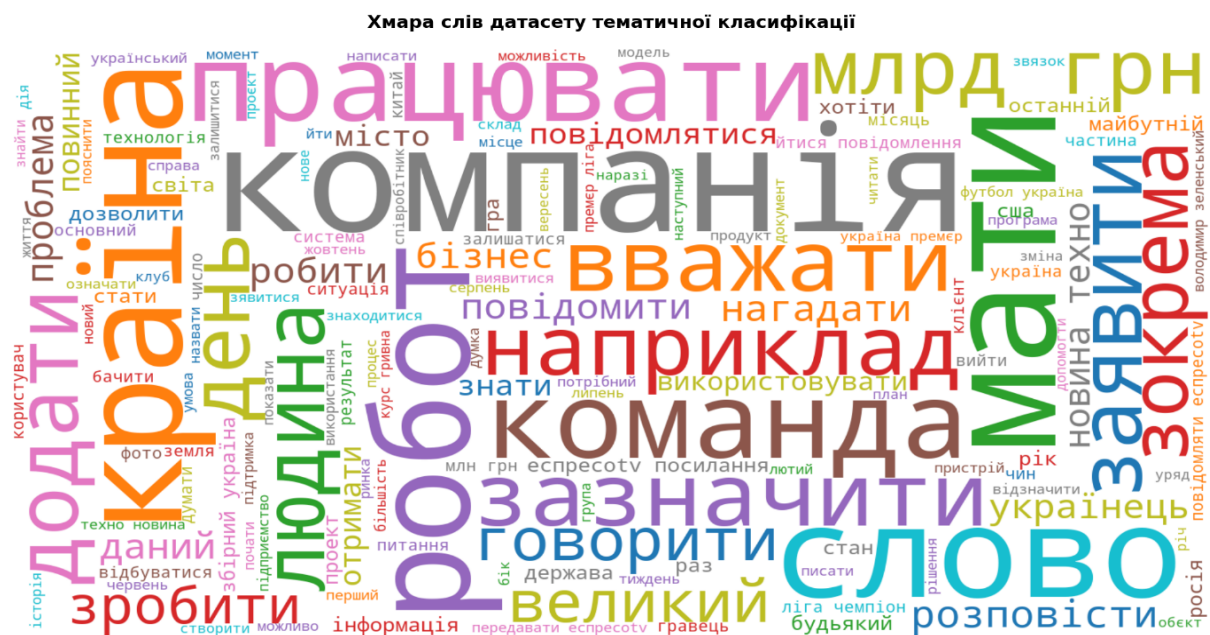


Рисунок 3.4 – Хмара найчастіших слів датасету тематичної класифікації після лематизації

3.3 Реалізація базових моделей та результати

Перший етап практичної реалізації передбачав побудову базових моделей на основі класичних алгоритмів машинного навчання з TF-IDF векторизацією. Метою цього кроку – встановити орієнтирні метрики якості, відносно яких оцінюватимуться складніші архітектури на базі трансформерів.

Для бінарної класифікації новин на категорії TRUE та FAKE реалізовано пайплайн із трьох послідовних етапів: попередня обробка тексту, TF-IDF векторизація та навчання логістичної регресії.

Векторизатор налаштовано зі словником у 30 000 ознак, урахуванням уніграм та біграм, фільтрацією рідкісних слів (`min_df=2`) та логарифмічним масштабуванням частот (`sublinear_tf=True`). Біграми дозволяють моделі вловлювати стійкі словосполучення, характерні для маніпулятивних текстів. Класифікатор `LogisticRegression` навчено з параметром регуляризації `C=2.0` – значення, дещо більше за одиницю, що дозволяє моделі краще адаптуватись до тренувальних даних без суттєвого ризику перенавчання. Оптимізатор `saga` обрано як найефективніший для розріджених текстових матриць великого розміру.

Результати оцінки моделі на тестовій вибірці наведено у таблиці 3.1.

Таблиця 3.1 – Метрики якості базової моделі (LogReg + TF-IDF)

Модель	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline LogReg	0.8408	0.8541	0.8219	0.8377	0.9216

Модель досягла точності 84.08% та F1-score 83.77%. Показник Precision дещо перевищує Recall, що свідчить про схильність класифікатора до «перестраховки» – він рідше помиляється при маркуванні новини як фейкової, проте пропускає близько 17.8% реальних фейків. Детальний розподіл передбачень наведено на рисунку 3.5, а висока якість ймовірнісного ранжування підтверджується значенням ROC-AUC = 0.9216 (рисунок 3.6).

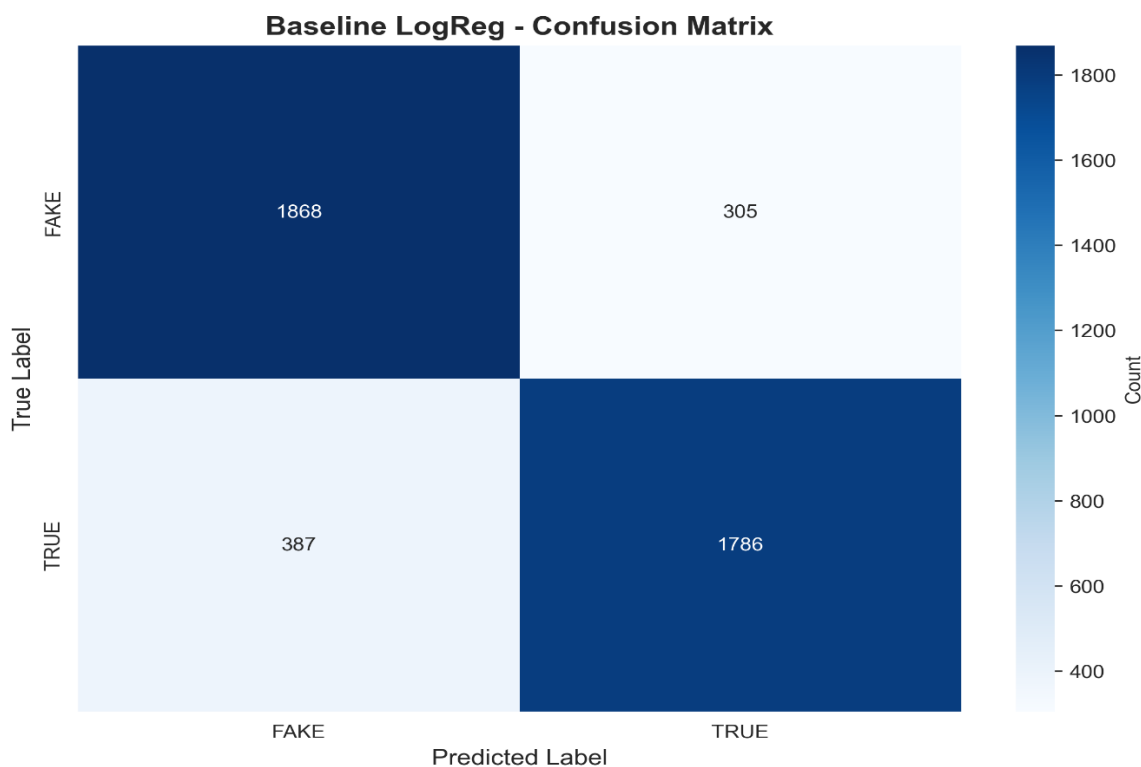


Рисунок 3.5 – Матриця помилок базової моделі логістичної регресії

Аналіз вагових коефіцієнтів моделі (рисунок 3.7) виявив специфічні зміщення у навчальній вибірці. Для класу FAKE модель спирається переважно на маркери каналів поширення («телеграм канал», «група поширити») та лексику фактчекерів («відеофейк»). Для класу TRUE найвагомішими виявились брендування медіа та лексика воєнних зведень. Це наочно демонструє

фундаментальне обмеження TF-IDF – алгоритм орієнтується на частотні теги джерел, повністю ігноруючи контекст і суть маніпуляції.

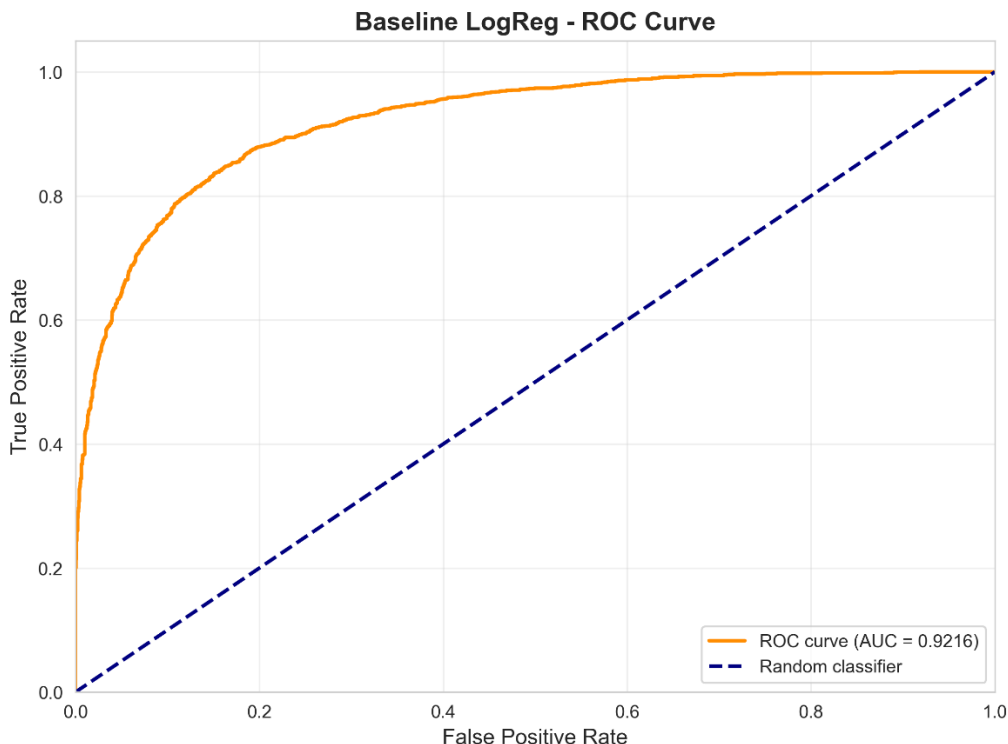


Рисунок 3.6 – ROC-крива базової моделі детекції фейків

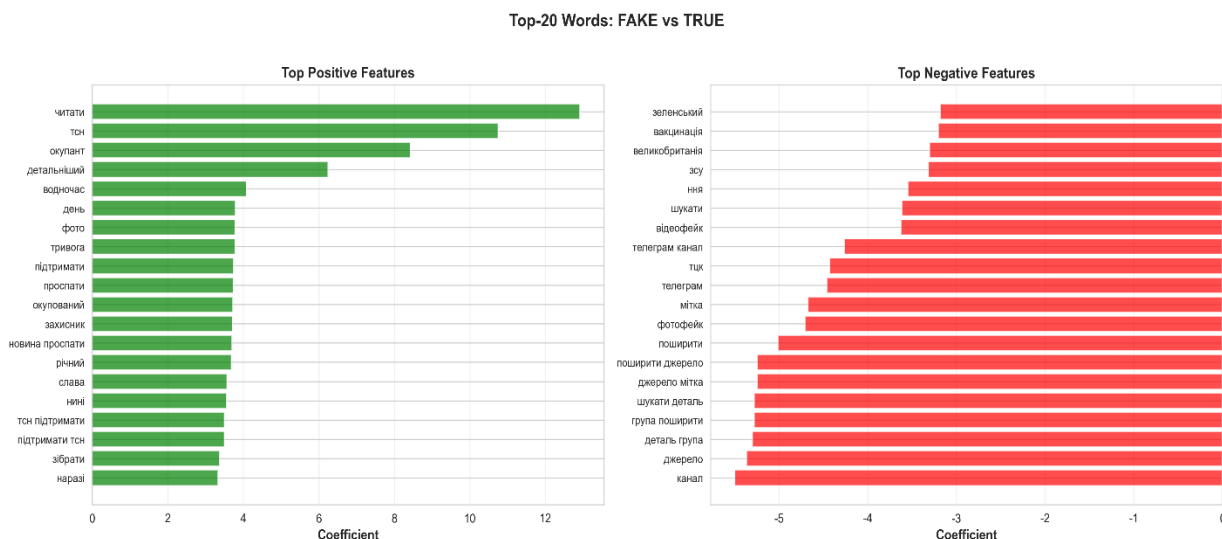


Рисунок 3.7 – Найбільш вагомні лексичні маркери для класів TRUE та FAKE

Для тематичної класифікації реалізовано аналогічний пайплайн на основі методу опорних векторів. Векторизатор налаштовано з жорсткішою фільтрацією шуму ($\text{min_df}=3$, $\text{max_df}=0.85$) порівняно з моделлю детекції фейків. Базовий класифікатор LinearSVC обгорнуто у CalibratedClassifierCV з 3-кратною крос-валідацією – це дозволило отримати калібровані ймовірності для кожної теми,

необхідні для відображення впевненості моделі у веб-інтерфейсі. Параметр `dual=False` обрано оскільки кількість зразків у датасеті перевищує кількість ознак.

Таблиця 3.2 – Метрики якості базової моделі LinearSVC (5 класів)

Категорія	Precision	Recall	F1–Score
бізнес	0.93	0.93	0.9318
новини	0.86	0.85	0.8552
політика	0.88	0.89	0.8840
спорт	0.99	0.99	0.9880
технології	0.95	0.94	0.9456
Macro Avg	0.92	0.92	0.9209

Загальна точність склала 92.13%. Найвищий показник зафіксовано для категорії «спорт» ($F1 = 0.9880$) завдяки вузькоспеціалізованій лексиці. Натомість категорії «новини» ($F1 = 0.8552$) та «політика» ($F1 = 0.8840$) показали найгірші результати. Аналіз матриці помилок (рисунк 3.8) виявив системну проблему: модель масово плутає тексти цих двох категорій – 135 випадків «новини → політика» та 112 випадків «політика → новини».

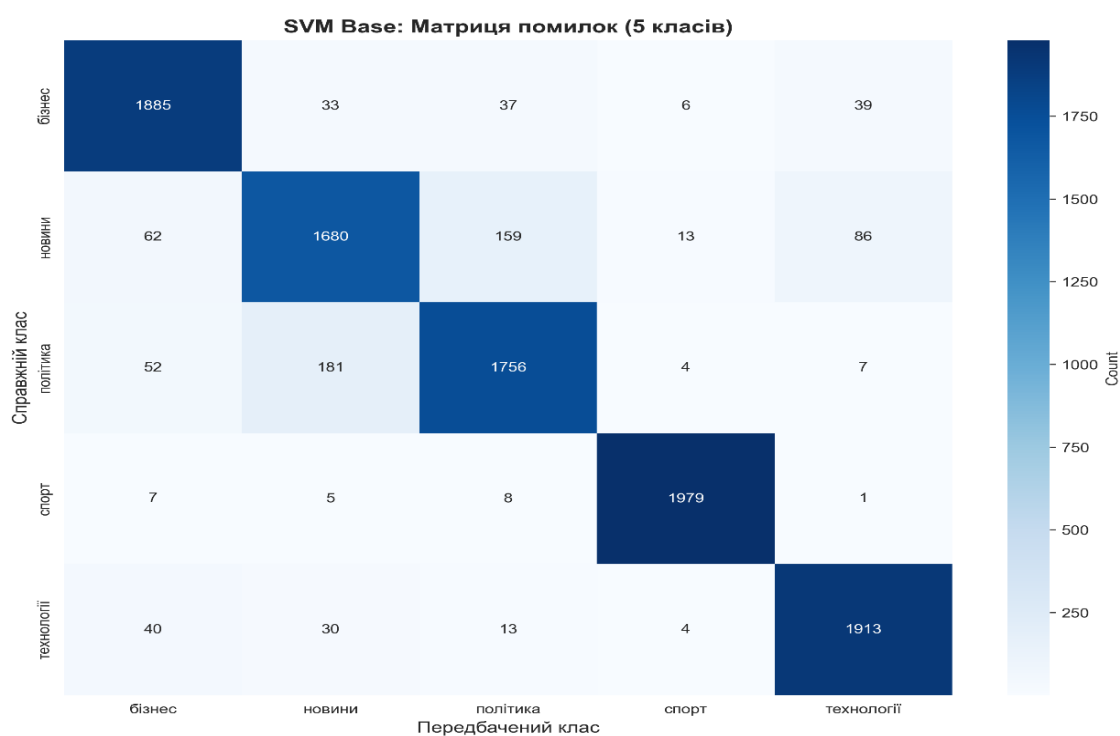


Рисунок 3.8 – Матриця помилок базової моделі LinearSVC

Ця проблема є концептуальною – категорія «новини» у зібраному корпусі є семантично розмитою та охоплює загальні матеріали, які за лексикою тісно перетинаються з політичними подіями. Оскільки базовий підхід TF-IDF оперує виключно частотністю слів і не розуміє контексту, він не здатен провести чітку межу між цими категоріями. Це обґрунтовує необхідність структурної оптимізації датасету, описаної у підрозділі 3.4.

Після видалення категорії «новини» та перенавчання на чотирьох класах модель SVM ExpB досягла точності 96.75% та Macro F1 = 0.9675, що підтверджує ефективність цього рішення (рисунк 3.9).

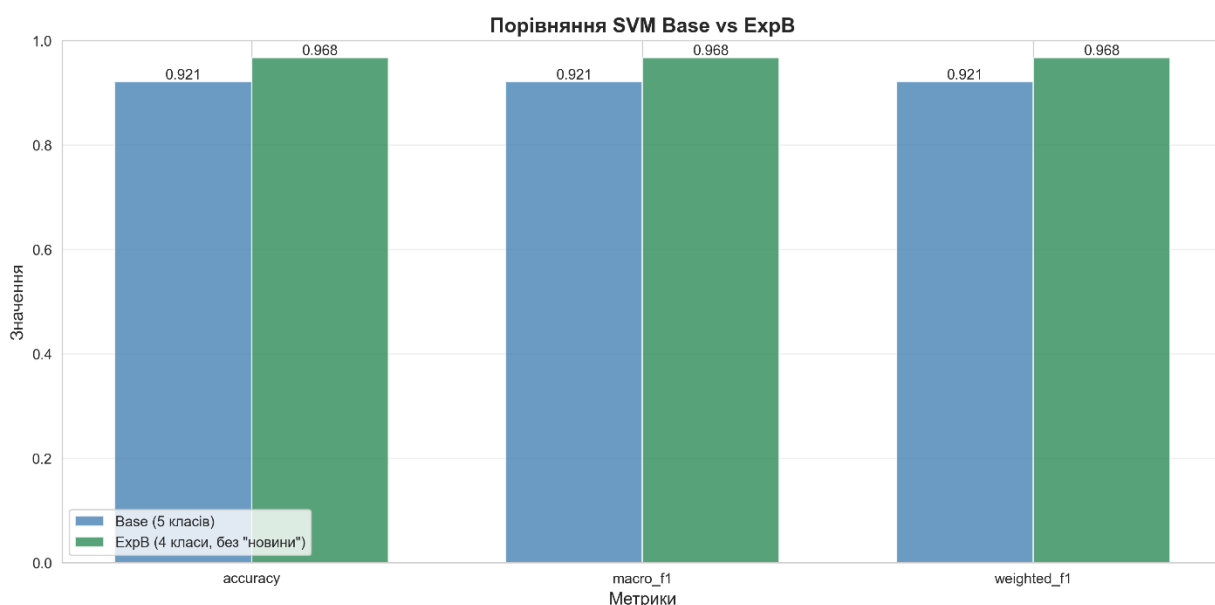


Рисунок 3.9 – Порівняння SVM Base vs SVM ExpB

3.4 Реалізація моделей на основі mBERT та результати

Другим етапом практичної реалізації стало дослідження нейромережових архітектур на базі багатомовного трансформера bert-base-multilingual-cased (mBERT). Головна мета – з'ясувати, наскільки контекстуальні представлення трансформера перевищують базовий підхід TF-IDF, та визначити оптимальну архітектуру класифікаційної голови.

Для забезпечення об'єктивності порівняння всі архітектури навчались за ідентичних умов із застосуванням стратегії повного донавчання (full fine-tuning), за якої оновлювались ваги як базового енкодера, так і класифікаційних шарів.

Вибір гіперпараметрів обґрунтовано нижче.

Гіперпараметри навчання моделі:

- 1) **MAX_LEN: 256** Лише 2.5% текстів у датасеті перевищують довжину у 256 токенів. Вибір такого ліміту є оптимальним, оскільки подальше збільшення призведе до непропорційного зростання споживання відеопам'яті (VRAM) через квадратичну складність механізму Self-Attention.
- 2) **BATCH_SIZE: 16** Це значення обрано як «золоту середину» для забезпечення стабільності градієнтів та дотримання лімітів доступної пам'яті графічного процесора.
- 3) **NUM_EPOCHS: 3** Класичний стандарт для донавчання (fine-tuning) моделей архітектури BERT. Більша кількість епох суттєво підвищує ризик перенавчання (overfitting) на специфічних даних.
- 4) **LEARNING_RATE: 1e-5** Знижена швидкість навчання допомагає стабілізувати процес збіжності моделі та запобігає «руйнуванню» (деградації) вже існуючих вагових коефіцієнтів, отриманих під час попереднього навчання.
- 5) **DROPOUT: 0.4** Використовується підвищений коефіцієнт дропауту для кращої регуляризації. Це критично важливо для запобігання перенавчанню, враховуючи роботу з відносно невеликим набором даних.
- 6) **WEIGHT_DECAY: 0.01** Застосовується L2-регуляризація в оптимізаторі AdamW для обмеження росту ваг та покращення узагальнюючої здатності моделі.
- 7) **WARMUP_RATIO: 0.1** 10% від загальної кількості кроків навчання виділяється на «лінійне прогрівання» (warmup). Це поступово збільшує learning rate на початку, що робить старт навчання менш агресивним.
- 8) **GRADIENT_CLIP: 1.0** Встановлено обмеження норми градієнтів, що дозволяє уникнути проблеми «вибухаючих градієнтів» та забезпечує стабільність обчислень протягом усього процесу навчання.

Для агрегації ознак, видобутих mBERT, реалізовано та досліджено три класифікаційні голови. Head A реалізує пряму лінійну проєкцію вектора [CLS]-

токена через шар Dropout та повнозв'язний шар Linear(768→2). Head B розширює цей підхід додатковим прихованим шаром із пакетною нормалізацією: Linear(768→512) → BatchNorm → ReLU → Dropout → Linear(512→2). Head C реалізує механізм зваженого усереднення прихованих станів усіх токенів послідовності – замість покладання виключно на [CLS]-токен модель навчається визначати внесок кожного слова у фінальне представлення.

Результати порівняння трьох архітектур на тестовій вибірці наведено у таблиці 3.3.

Таблиця 3.3 – Порівняння класифікаційних голів mBERT (детекція фейків)

Голова	Accuracy	Precision	Recall	F1	ROC–AUC
Head A	0.9243	0.9250	0.9243	0.9242	0.9829
Head B	0.9270	0.9293	0.9270	0.9269	0.9823
Head C	0.9295	0.9306	0.9295	0.9294	0.9827

Всі три архітектури суттєво перевершують базову логістичну регресію – приріст F1 становить від +9.2 до +9.7 процентних пунктів. Найвищий показник F1 демонструє Head C (0.9294), що підтверджує перевагу зваженого усереднення всіх токенів над використанням лише [CLS]-токена. Водночас різниця між головами є незначною – у межах 0.5 п.п., що доводить ключову тезу: основний внесок у якість розпізнавання фейків вносить сам mBERT-енкодер, а вибір топології класифікаційного шару є лише інструментом тонкого налаштування.

Динаміка навчання всіх трьох голів наведена на рисунку 3.10 – криві функції втрат демонструють стабільне зниження без ознак перенавчання.

Матриці помилок та ROC-криві для всіх архітектур наведено на рисунку 3.11. Спільною тенденцією є переважання хибнопозитивних помилок над хибнонегативними – система частіше маркує правдиву новину як фейкову, ніж навпаки. У контексті фактчекінгу така поведінка є прийнятною, оскільки мінімізує ризик пропустити реальну дезінформацію.

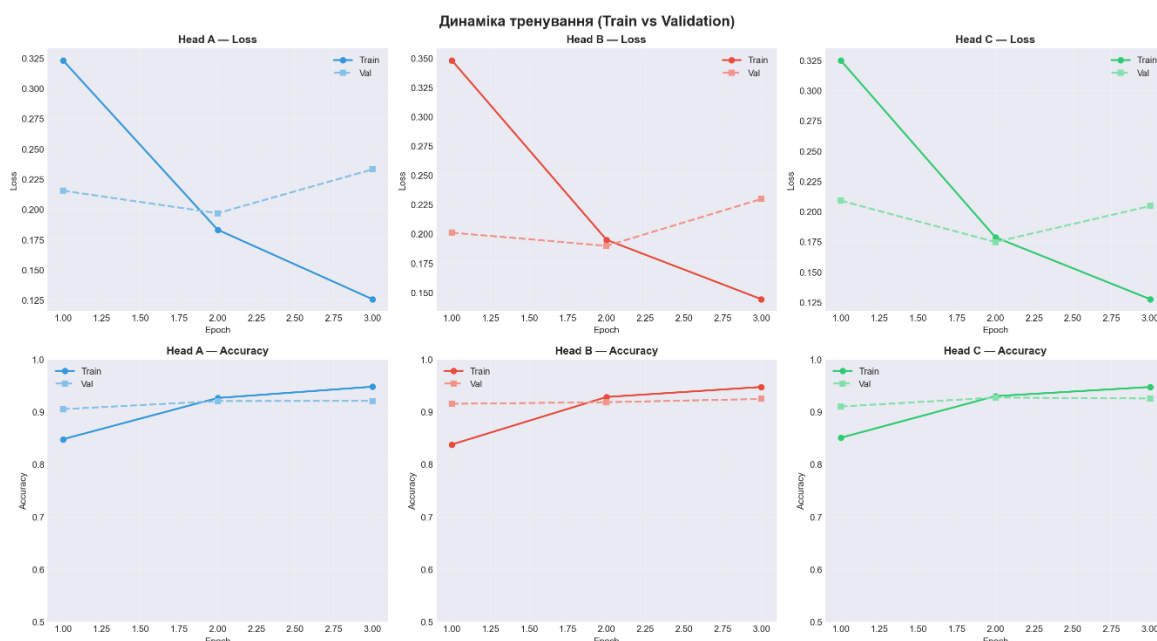


Рисунок 3.10 – Динаміка навчання та порівняння голів mBERT

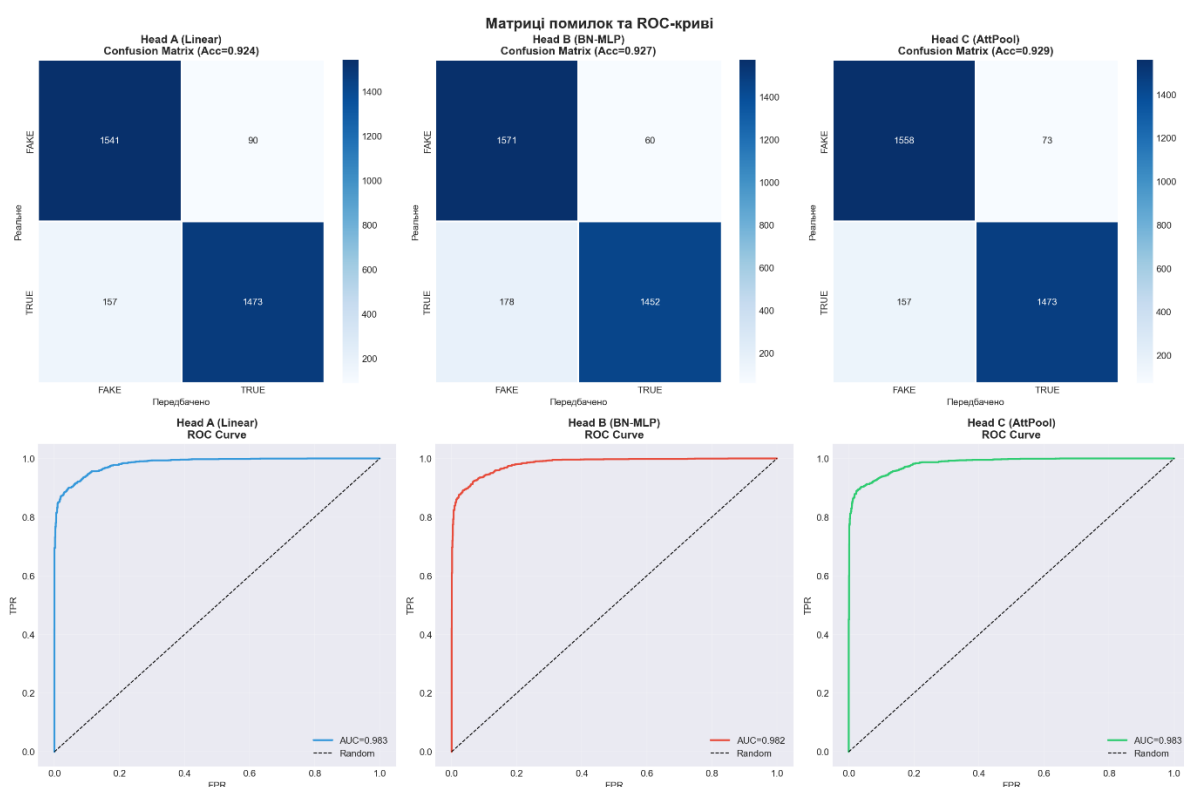


Рисунок 3.11 – Матриці помилок та ROC-криві для всіх архітектур

У реальних умовах датасети фактчекінгу майже ніколи не є збалансованими. Для дослідження стійкості моделей до цієї проблеми сформовано датасет зі співвідношенням TRUE:FAKE $\approx 2:1$ (10 869 правдивих та 5 434 фейкових новини). Результати наведено у таблиці 3.4.

Таблиця 3.4 – Результати Експерименту А (дисбаланс TRUE:FAKE $\approx$ 2:1)

Голова	Accuracy	F1	ROC-AUC	Δ F1 vs збалансований
Head A	0.9203	0.9206	0.9811	-0.0037
Head B	0.9272	0.9274	0.9807	+0.0005
Head C	0.9244	0.9244	0.9814	-0.0050

Навіть при дворазовому дисбалансі класів всі архітектури зберігають F1 > 92%. Цікавою аномалією є поведінка Head B – її F1 практично не змінився завдяки стабілізуючому ефекту шару BatchNorm. Значення ROC-AUC знизились лише на 0.002, що підтверджує нечутливість цієї метрики до дисбалансу класів.

Для тематичної класифікації застосовано архітектуру Head A з гіперпараметрами, зафіксованими у попередніх експериментах. На першому етапі модель навчалась на повному датасеті з п'ятьма категоріями та досягла точності 94.59% і F1 = 0.9459. Проте аналіз матриці помилок (рисунок 3.12) виявив збереження критичної проблеми – модель систематично плутає категорії «новини» та «політика»: 119 помилок у напрямку «політика $\rightarrow$ новини» та 98 у зворотному.

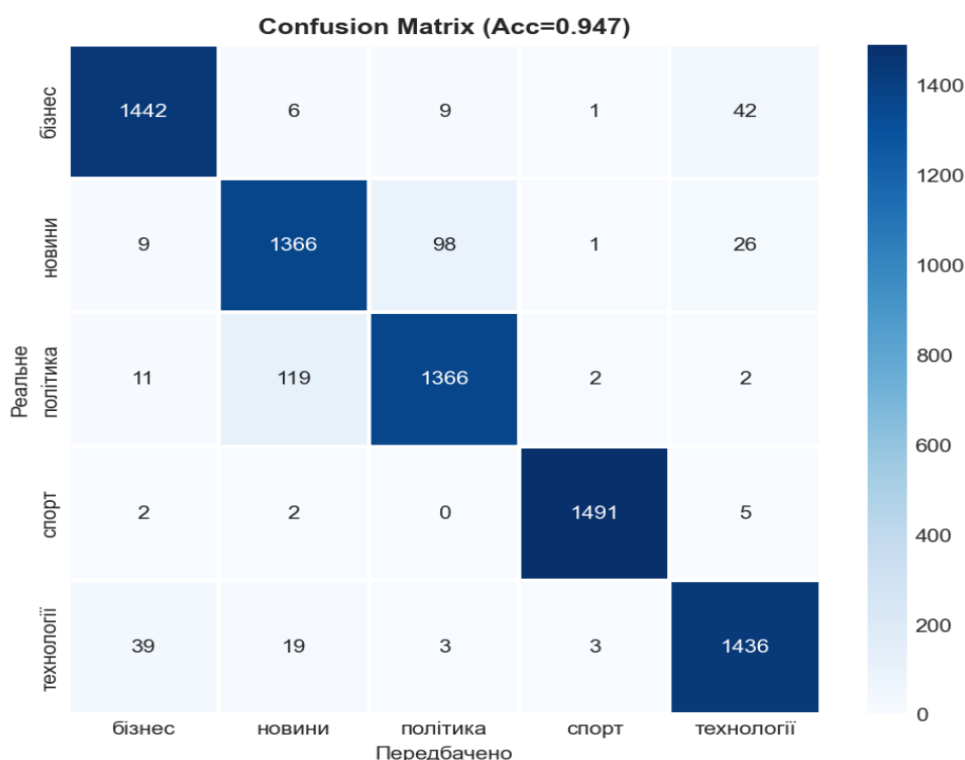


Рисунок 3.12 – Матриця помилок базової моделі mBERT (5 класів)

Це емпірично доводить що проблема полягає не у слабкості алгоритму, а в самій природі даних – обидві рубрики оперують однаковими сутностями і об'єктивної межі між ними не існує. Після повного видалення категорії «новини» з датасету (Експеримент В) модель перенавчено на чотирьох класах. Результати порівняння конфігурацій наведено у таблиці 3.5.

Таблиця 3.5 – Результати mBERT для тематичної класифікації

Конфігурація	Accuracy	Macro F1	ROC-AUC
BERT Topic Base (5 класів)	0.9459	0.9459	0.9949
BERT Topic ExpV (4 класи)	0.9772	0.9772	0.9983

Приріст якості після структурної оптимізації датасету склав +3.13 п.п. Криві навчання (рисунок 3.13) демонструють стабільну збіжність без ознак перенавчання, а матриця помилок має яскраво виражену діагональ – проблема плутанини між категоріями повністю зникла.

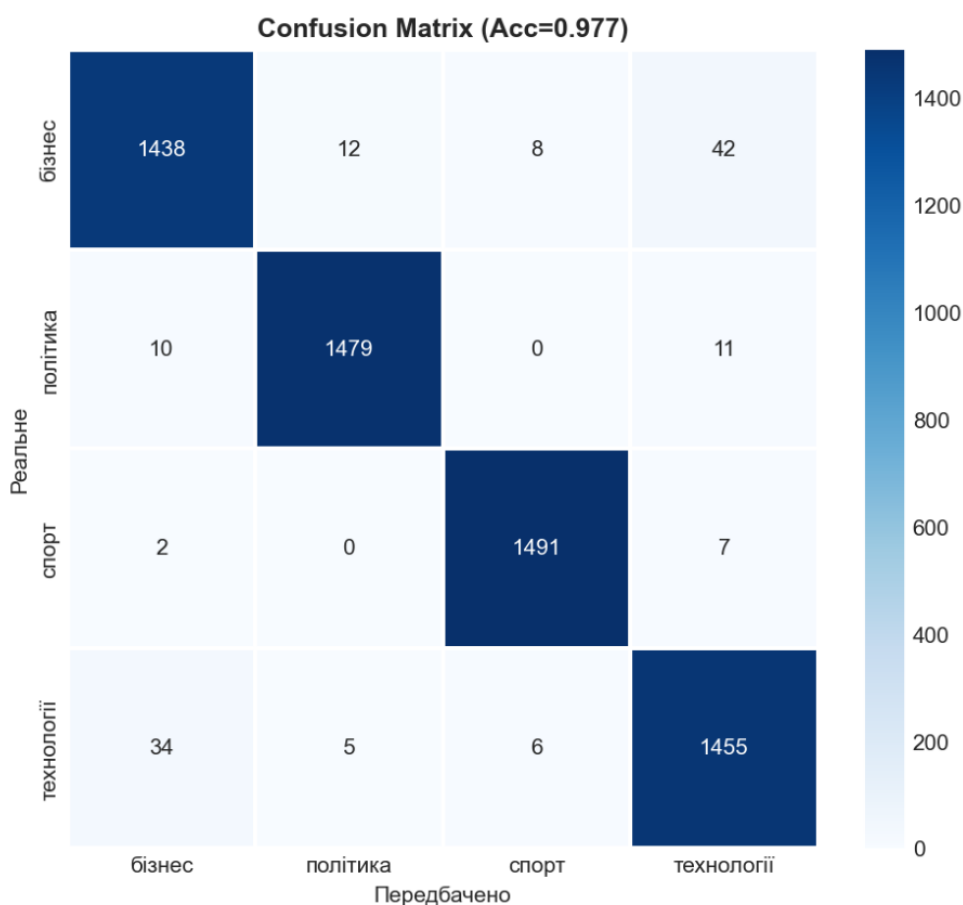


Рисунок 3.13 – Криві навчання та матриця помилок BERT Topic ExpV

З огляду на досягнуту точність 97.72% модель BERT Topic ExpV обрано як фінальну для модуля тематичної категоризації.

3.5 Розгортання вебдодатка

Через технічні обмеження безкоштовних хмарних платформ розгортання реалізовано за розподіленою схемою на базі Hugging Face – безкоштовної платформи для публікації моделей та застосунків машинного навчання.

Для обходу ліміту пам'яті створено два окремих репозиторії. Перший – Model Repository (yatsenq/news-detector-weights) – слугує сховищем для важких файлів: ваг неймереж BERT, серіалізованих моделей логістичної регресії та SVM і токенизаторів [26]. Другий – Hugging Face Space (yatsenq/coursework-fake-news-detector) – містить лише програмний код Flask, шаблони інтерфейсу та конфігураційні файли [27]. Повний вихідний код проєкту, конфігураційні файли та інструкції з розгортання опубліковані у відкритому репозиторії на платформі GitHub [28].

Процес розгортання відбувається автоматично після завантаження коду та складається з чотирьох етапів. На етапі збірки сервер читає Dockerfile та requirements.txt і встановлює всі залежності. На етапі запуску стартує WSGI-сервер Gunicorn. Під час першого звернення до неймережі функція load_selected_models автоматично завантажує ваги моделей з Model Repository у внутрішній кеш сервера. При старті застосунку також виконується автоматична міграція бази даних – перевіряється структура таблиць і додаються нові колонки якщо вони з'явилися у коді. Для стабільної роботи за проксі-сервером використовується ProxyFix, що забезпечує коректну передачу HTTPS-заголовків для сесій користувачів. Розгорнутий застосунок доступний на платформі Hugging Face Spaces за адресою yatsenq/coursework-fake-news-detector [29].

3.6 Огляд роботи системи

Розроблений вебдодаток Fake News Detector реалізує повний цикл аналізу новин – від введення тексту до отримання детального звіту. Інтерфейс виконано у темній кольоровій схемі та розроблено з урахуванням зручності використання без технічної підготовки.

Головна сторінка виконує роль інформаційного хабу платформи (рисунок 3.14). У верхній частині розміщено заголовок з описом ключових можливостей системи та кнопка переходу до аналізу. Нижче відображається блок глобальної аналітики – інтерактивна кругова діаграма розподілу перевірених новин за тематичними категоріями та лічильники загальної кількості перевірок і виявлених фейків за поточну добу.

Основний робочий простір користувача складається з двох панелей. Ліва панель містить текстове поле для введення новини або URL-посилання, кнопки «Аналіз» та «Очистити», а також блок швидкого завантаження прикладів – «Фейк (StopFake)» та «Правда (Укрінформ)» – які автоматично завантажують реальні новини з відповідних джерел

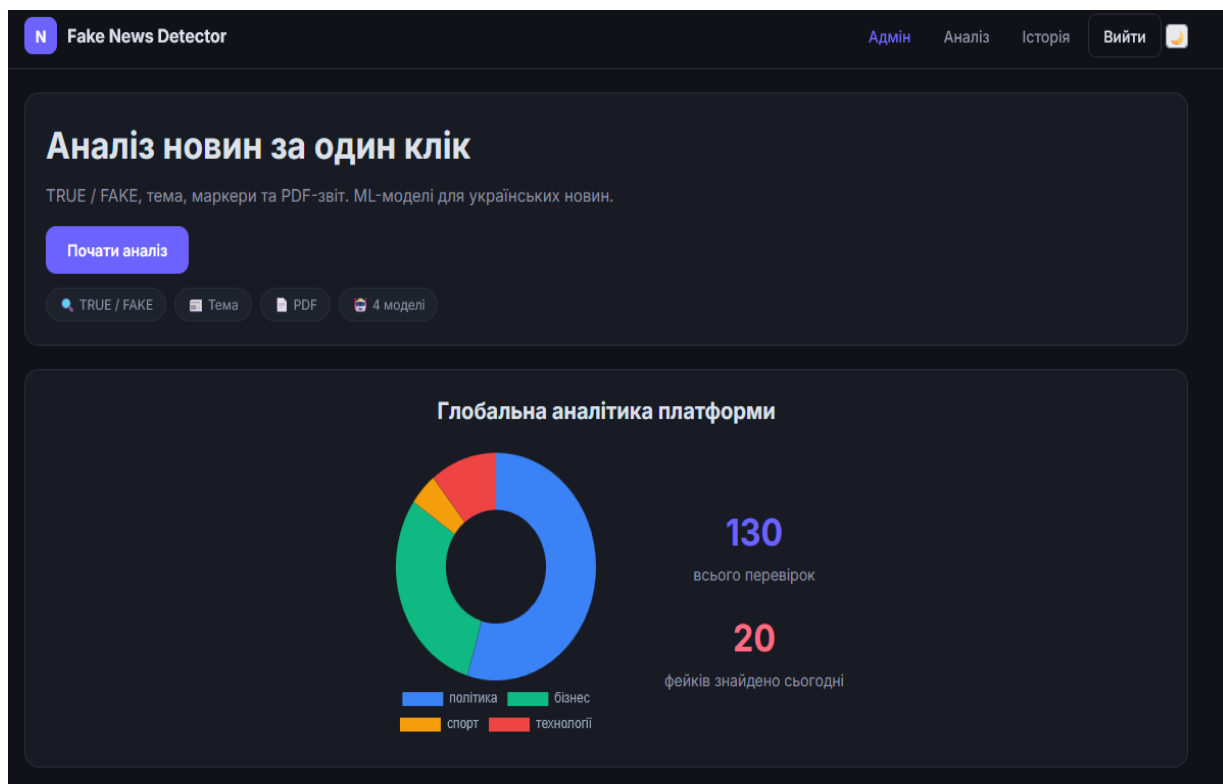


Рисунок 3.14 – Головна сторінка з глобальною аналітикою платформи

Після натискання кнопки «Аналіз» права панель динамічно оновлюється з результатами. У разі виявлення фейку відображається червона мітка FAKE із загальним відсотком впевненості – у наведеному прикладі 92.9% (рисунок 3.16). Нижче розміщено горизонтальні шкали з розподілом ймовірностей між класами TRUE та FAKE, кнопку розгортання детальних результатів кожної моделі, кнопку

завантаження PDF-звіту та блок зворотного зв'язку з кнопками «Згоден» та «Не згоден».

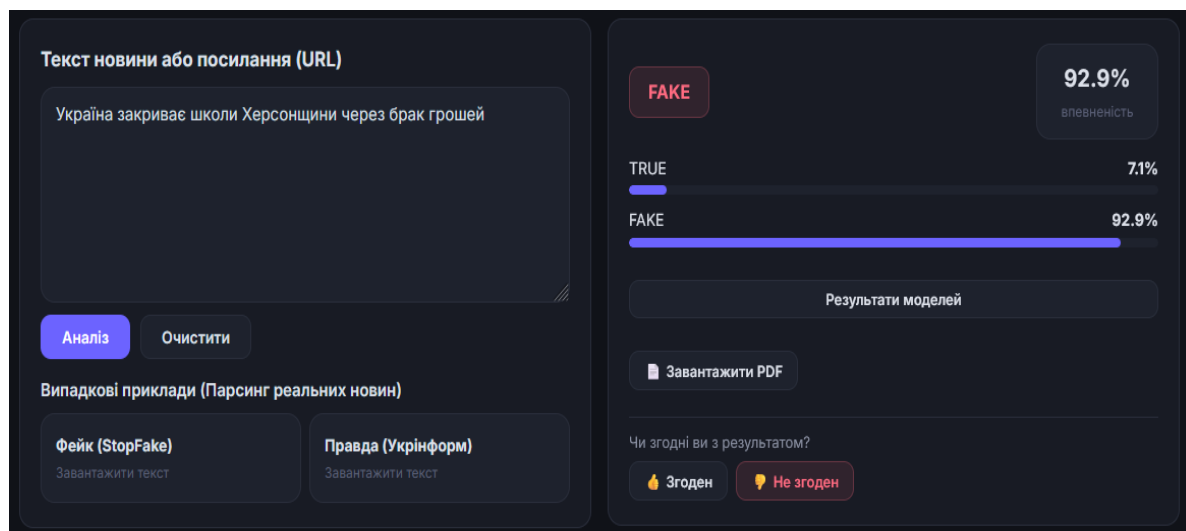


Рисунок 3.15 – Результат аналізу: виявлення фейкової новини

Для правдивої новини інтерфейс відображає зелену мітку TRUE з відповідним розподілом ймовірностей (рисунок 3.16). У наведеному прикладі модель визначила текст як достовірний з впевненістю 90.0%.

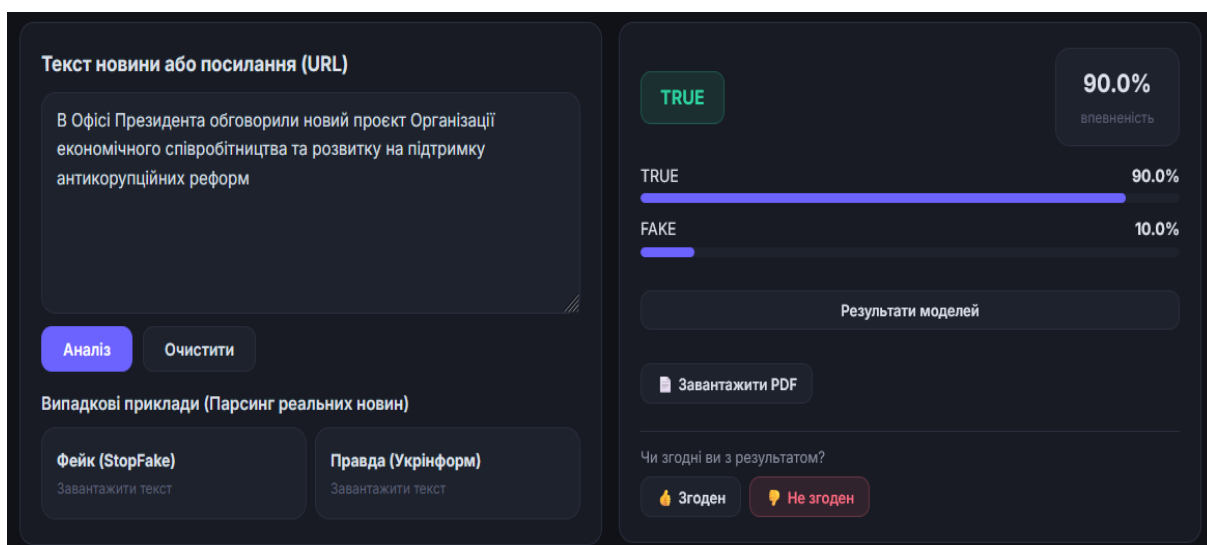


Рисунок 3.16 – Результат аналізу: правдива новина

Сторінка історії доступна авторизованим користувачам та відображає всі попередні перевірки у вигляді карток (рисунок 3.17). Кожна картка містить мітку результату з кольоровою індикацією, дату та час перевірки, повний текст новини, визначену тематичну категорію та розподіл ймовірностей. Для кожного запису

доступне завантаження PDF-звіту. Сторінка також підтримує пошук за текстом або темою та можливість повного очищення історії.

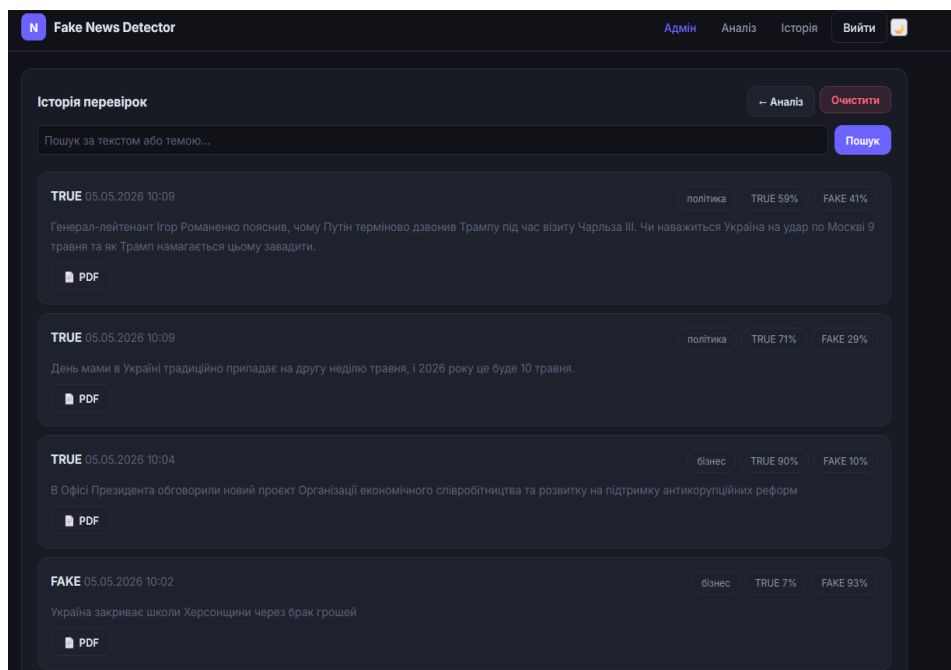


Рисунок 3.17 – Сторінка історії перевірок користувача

Система надає можливість завантаження структурованого звіту у форматі PDF (рисунок 3.18). Звіт містить дату аналізу, повний вхідний текст новини, таблицю результатів детекції фейку з ймовірностями TRUE та FALSE, результати тематичної класифікації з впевненістю та таблицю ключових слів-маркерів.

Звіт аналізу новини

Дата: 05.05.2026 10:10

Вхідний текст

Генерал-лейтенант Ігор Романенко пояснив, чому Путін терміново дзвонив Трампу під час візиту Чарльза III. Чи наважиться Україна на удар по Москві 9 травня та як Трамп намагається цьому завадити.

Результат fake detection

Мітка	TRUE
Ймовірність TRUE	58.53%
Ймовірність FALSE	41.47%

Тематична класифікація

Категорія	політика
Впевненість	93.24%

Ключові слова-маркери

Слово	Вага
трамп	0.5794
москва	0.2395
путін	0.1329
завадити	0.1010
візит	0.0958
пояснити	0.0719
чарльз	0.0298

Рисунок 3.18 – Згенерований PDF-звіт аналізу новини

Для адміністраторів системи реалізовано окрему панель керування (рисунок 3.19). Вона відображає загальну статистику платформи – кількість зареєстрованих користувачів та загальну кількість перевірок – а також таблицю користувачів з інформацією про статус облікового запису та час останньої активності.

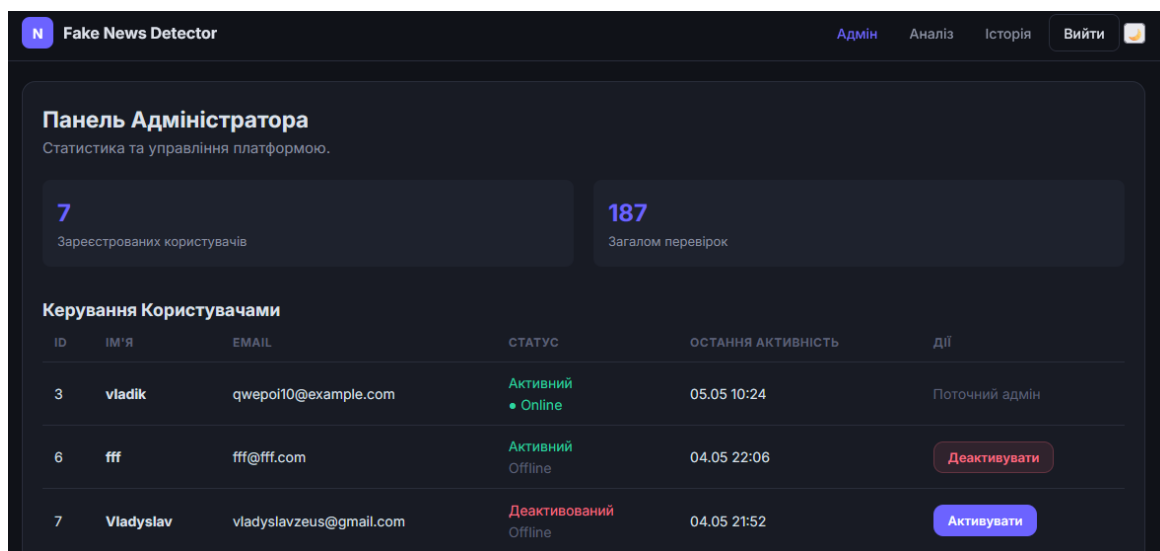


Рисунок 3.19 – Панель адміністратора

3.7 Тестування системи та аналіз помилок

Для оцінки практичної ефективності розробленої системи проведено функціональне тестування на п'яти тестових прикладах, що охоплюють різні сценарії використання: очевидні фейки, достовірні новини, граничні випадки та тексти з маніпулятивними елементами.

1. Очевидна дезінформація.

Текст: «Україна закриває школи Херсонщини через брак грошей»

Таблиця 3.6 – Результати тесту 1

Модель	Завдання	Результат	Впевненість
Baseline LogReg	Детекція фейку	FAKE	86.0%
BERT Head C	Детекція фейку	FAKE	99.9%
SVM ExpB	Тематика	бізнес	86.5%
BERT Topic ExpB	Тематика	бізнес	100.0%

Обидві моделі детекції одноставно класифікували текст як фейковий. BERT Head C демонструє значно вищу впевненість (99.9% проти 86.0%), що свідчить про

кращу здатність трансформера розпізнавати маніпулятивні конструкції. Тематичні моделі визначили категорію «бізнес», що є частково коректним – новина стосується фінансування освіти.

2. Хибнопозитивне спрацювання (False Positive)

Текст: *«В Офісі Президента обговорили новий проєкт Організації економічного співробітництва та розвитку на підтримку антикорупційних реформ»*

Таблиця 3.7 – Результати тесту 2

Модель	Завдання	Результат	Впевненість
Baseline LogReg	Детекція фейку	TRUE	90.0%
BERT Head C	Детекція фейку	FAKE	57.4%
SVM ExpB	Тематика	бізнес	82.8%
BERT Topic ExpB	Тематика	бізнес	100.0%

BERT Head C помилково класифікує достовірну новину як фейкову з впевненістю 57.4%. Низьке значення впевненості вказує на граничний випадок – офіційний стиль тексту з аббревіатурами та назвами організацій активує патерни характерні для маніпулятивних текстів у тренувальній вибірці. LogReg натомість коректно визначає текст як TRUE спираючись на лексику офіційних повідомлень.

3. Хибнонегативне спрацювання (False Negative)

Текст: *«Секретна вакцина від всеосяжного вірусу вийшла в Україні: експерти запевняють, що вона вже врятувала тисячі!»*

Таблиця 3.8 – Результати тесту 3

Модель	Завдання	Результат	Впевненість
Baseline LogReg	Детекція фейку	FAKE	96.8%
BERT Head C	Детекція фейку	TRUE	72.8%
SVM ExpB	Тематика	технології	61.9%
BERT Topic ExpB	Тематика	технології	55.0%

BERT Head C класифікує очевидний фейк як TRUE з впевненістю 72.8%, тоді як LogReg впорався коректно (FAKE, 96.8%), спираючись на характерну лексику сенсаційних заголовків. Цей випадок демонструє що трансформерна модель попри загальну перевагу може поступатись базовій у специфічних сценаріях із клікбейтними заголовками.

4. Тематична класифікація

Для оцінки модуля тематичної класифікації подано чотири тексти з різних предметних областей. Результати наведено у таблиці 3.9.

Таблиця 3.9 – Результати тесту 4

Текст	Очікувана тема	SVM ExpB	BERT Topic ExpB
Про акредитацію спортсменів	спорт	спорт	спорт
Про сканування паспортів у магазинах	бізнес	бізнес	бізнес
Про стандарт ШІ-моделей у держорганах	технології	технології	технології
Про відстрочення виборів до місцевих рад	політика	політика	бізнес

5. Змішаний текст із маніпулятивним елементом

Текст: «Міністр освіти України дійсно затвердив оновлений навчальний план для 11-го класу, де вперше введено окремий модуль з цифрової грамотності. Однак у деяких ЗМІ заявилося, що цей план зобов'язує школи автоматично встановлювати соціальні медіа-акаунти для всіх учнів, що є маніпулятивним домислом без підтвердження в документі.»

Текст містить реальний факт – затвердження оновленого навчального плану для 11-го класу. Це офіційна інформація, яка може бути підтверджена відповідними документами або заявами освітніх органів. Водночас у цьому ж тексті додається маніпулятивний домисел про нібито обов'язкове встановлення соціальних мереж

для всіх учнів, що не має підтвердження і виглядає як перебільшення або викривлення факту.

Таблиця 3.10 – Результати тесту 5

Модель	Завдання	Результат	Впевненість
Baseline LogReg	Детекція фейку	FAKE	55.3%
BERT Head C	Детекція фейку	TRUE	98.4%
SVM ExpB	Тематика	бізнес	91.5%
BERT Topic ExpB	Тематика	технології	99.6%

Це найскладніший тестовий випадок. BERT Head C з впевненістю 98.0% класифікує текст як TRUE не розпізнавши маніпулятивну частину. LogReg визначає текст як FAKE з низькою впевненістю 55.3%. Жодна з моделей не впоралась коректно – це принципове обмеження підходу бінарної класифікації який не здатен виявляти часткову маніпуляцію всередині тексту.

За результатами тестування виявлено кілька системних закономірностей. По-перше, BERT Head C загалом перевершує LogReg але програє йому на текстах із клікбейтною лексикою та сенсаційними заголовками. По-друге, BERT Topic ExpB має систематичне зміщення у бік категорії «бізнес» для політичних текстів із лексикою державного управління – у таких випадках SVM ExpB демонструє вищу точність. По-третє, обидві моделі детекції мають труднощі з текстами що містять одночасно достовірні факти та маніпулятивні твердження. Ці обмеження відображено у застереженні веб-інтерфейсу про те що результати є автоматичними прогнозами та потребують додаткової перевірки користувачем.

ВИСНОВКИ

У курсовій роботі було проведено комплексне дослідження методів машинного навчання для автоматизації верифікації інформації та розроблено інтелектуальну систему для виявлення фейкових новин. Завдяки детальному аналізу існуючих рішень (StopFake, VoxCheck та ін.) та дослідженню сучасних NLP-технологій, було створено програмний продукт, що поєднує високу точність неймережевих моделей із доступністю через вебінтерфейс.

Застосування багатомовної моделі mBERT дозволило системі ефективно обробляти специфіку української мови, зокрема її складну морфологію та контекстуальні зв'язки. У ході експериментів було доведено, що використання трансформерних архітектур із механізмом Attention Pooling забезпечує значну перевагу над традиційними статистичними методами, підвищуючи точність класифікації на понад 9%.

Розгортання системи на базі фреймворку Flask та платформи Hugging Face Spaces зробило інструмент легкодоступним для широкого кола користувачів. Тепер для перевірки новини достатньо завантажити текст або URL-посилання, після чого алгоритм у реальному часі надає об'єктивну оцінку достовірності та визначає тематичну категорію матеріалу.

У порівнянні з наявними аналогами, розроблена система виділяється здатністю паралельно виконувати детекцію маніпуляцій та тематичну класифікацію, надаючи користувачеві вичерпний аналітичний звіт у форматі PDF. Платформа успішно вирішує проблему масштабованості фактчекінгу, автоматизуючи первинну фільтрацію інформаційних потоків.

Створений вебзастосунок є безпечним та надійним інструментом, що може бути використаний у професійній діяльності журналістів, роботі медіа-аналітиків та для підвищення індивідуальної медіаграмотності громадян в умовах активних інформаційних воєн.

У майбутньому систему можна розширити шляхом впровадження механізмів інтерпретації рішень, що дозволить візуалізувати конкретні фрагменти тексту, які

модель вважає маніпулятивними. Також перспективним є додавання модулів для аналізу мультимедійного контенту та інтеграція з API популярних соціальних мереж для моніторингу дезінформації в режимі реального часу.

Отже, в умовах глобальної цифровізації та зростання обсягів пропаганди, інтелектуальні системи на основі трансформерних моделей відіграватимуть вирішальну роль у захисті інформаційної безпеки суспільства.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Reuters Institute for the Study of Journalism. Digital News Report 2023. University of Oxford, 2023. [Електронний ресурс] – Режим доступу: https://reutersinstitute.politics.ox.ac.uk/sites/default/files/2023-06/Digital_News_Report_2023.pdf
2. Google Fact Check Explorer. Google News Initiative. [Електронний ресурс] – Режим доступу: <https://toolbox.google.com/factcheck/explorer>
3. Fact Check Tools API. Google Developers. [Електронний ресурс] – Режим доступу: <https://developers.google.com/fact-check/tools/api>
4. StopFake. Про проєкт StopFake: верифікація фактів та боротьба з дезінформацією. [Електронний ресурс] – Режим доступу: <https://www.stopfake.org/uk/pro-nas/>
5. VoxCheck. Проєкт перевірки фактів VoxUkraine. [Електронний ресурс] – Режим доступу: <https://voxukraine.org/uk/voxcheck/>
6. FakeCheck.in.ua. Калькулятор фейкових новин. [Електронний ресурс] – Режим доступу: <https://fakecheck.in.ua/>
7. Gwara Media. Перевірка разом: Telegram-бот для фактчекінгу. [Електронний ресурс] – Режим доступу: <https://gwaramedia.com/perevirka-razom-smozhemo/>
8. PolitiFact. Fact-checking U.S. politics. [Електронний ресурс] – Режим доступу: <https://www.politifact.com/>
9. Fake News Research. Simon Fraser University. [Електронний ресурс] – Режим доступу: <http://fakenews.research.sfu.ca/>
10. Shu K., Sliva A., Wang S., Tang J., Liu H. Fake News Detection on Social Media: A Data Mining Perspective. [Електронний ресурс] – Режим доступу: <https://arxiv.org/abs/1708.01967>
11. Cortes C., Vapnik V. Support-Vector Networks. // Machine Learning. [Електронний ресурс] – Режим доступу: <https://link.springer.com/content/pdf/10.1007/BF00994018.pdf>

12. Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need. [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1706.03762>
13. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1810.04805>
14. Pires T., Schlinger E., Garrette D. How Multilingual is Multilingual BERT? [Электронный ресурс] – Режим доступа: <https://aclanthology.org/P19-1493/>
15. Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. [Электронный ресурс] – Режим доступа: https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf?utm_content=buffer79b4
16. Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. [Электронный ресурс] – Режим доступа: <https://dl.acm.org/doi/10.5555/3045118.3045167>
17. Ba J. L., Kiros J. R., Hinton G. E. Layer Normalization. [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1607.06450>
18. Harris C. R. et al. Array programming with NumPy. [Электронный ресурс] – Режим доступа: <https://www.nature.com/articles/s41586-020-2649-2>
19. Waskom M. L. seaborn: statistical data visualization. [Электронный ресурс] – Режим доступа: semanticscholar.org/paper/Seaborn:-Statistical-Data-Visualization-Waskom/9f1b79d77201016a2579c013c5d490a12ab596b7
20. Fabian Pedregosa, Scikit-learn: Machine Learning in Python. [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1201.0490>
21. PyMorphy3 documentation. [Электронный ресурс] – Режим доступа: <https://pymorphy3.readthedocs.io/>
22. Paszke A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. [Электронный ресурс] – Режим доступа: <https://arxiv.org/abs/1912.01703>

- 23.Flask Documentation. [Електронний ресурс] – Режим доступу:
<https://flask.palletsprojects.com/>
- 24.ReportLab PDF Library. [Електронний ресурс] – Режим доступу:
<https://www.reportlab.com/>
- 25.Gunicorn: Python WSGI HTTP Server. [Електронний ресурс] – Режим доступу: <https://gunicorn.org/>
- 26.Репозиторій моделей на Hugging Face. [Електронний ресурс] – Режим доступу: <https://huggingface.co/yatsenq/news-detector-weights/tree/main>
- 27.Репозиторій програмного коду на Hugging Face. [Електронний ресурс] – Режим доступу: <https://huggingface.co/spaces/yatsenq/coursework-fake-news-detector/tree/main>
- 28.Репозиторій програмного коду "Fake News Detector" на GitHub. [Електронний ресурс] – Режим доступу:
<https://github.com/yatsenq/Course-Work>
- 29.Розгорнутий застосунок на платформі Hugging Face Spaces. [Електронний ресурс] – Режим доступу:
<https://huggingface.co/spaces/yatsenq/coursework-fake-news-detector>